第一章：网络传输基础知识

一、核心概念

网络、互联网与 Web 的区别

【网络】：最广义的概念，如电网、神经网络等。
【互联网/Internet】：由许多小网络互联而成的广域网，Internet 是最大的互联网。主要由计算机和电缆组成。
【Web/万维网】：是 Internet 上的一个"应用层服务"。它用超文本（网页）将信息链接起来。除了 Web，QQ、Email 等也是 Internet 上的服务。
【概念辨析】：Web 不等于 Internet，Web 只是 Internet 众多应用层服务中的一种。

C/S 模型与 P2P 模型

【C/S（客户/服务器）模式】：客户端请求，服务器响应。服务器必须一直在固定的 IP 和熟知的端口上开机等待；客户端之间不能直接通信。
【P2P（点对点）模式】：并非一种新的网络物理结构，而是一种应用模式。每个节点既是客户也是服务器，直接通信，不依赖中心服务器。

代理服务器（Proxy）

【作用】：充当中间人。可以绕开防火墙、检查数据保密性、通过缓存降低网络流量并提升响应速度。【缺点】：无法应对所有协议。
【层级对比】：这里注意，防火墙通常工作在网络层（网际层）或传输层，基于 IP 和端口过滤数据包；
而代理服务器通常工作在应用层（如 HTTPProxy）或传输层（如 Socks 代理），能够解析并理解具体的应用层协议内容。

二、数据交换技术

电路交换（类似打电话）

【过程】：线路建立->数据传输（全双工）->电路拆除。
【特点】：通信前必须建立专用的物理链路，传输期间独占该链路。实时性好，但线路利用率低。

报文交换（类似寄一整箱快递）

【过程】：存储-转发。结点接收整个报文，缓存下来，查看目的地址，再发给下一个结点。
【特点】：不需要预先建立专用链路。缺点是需要将完整报文存下来再转发，时延较大，不适合高实时性通信。

分组交换/包交换（类似把一车苹果拆成一箱箱寄）

【核心】：属于"存储/转发"，但把报文划分成一个个小的"分组"，每个分组独立转发。
【两种子模式】：(1)数据报交换：每个分组都当作独立的"小报文"处理，各自寻找路由，可能乱序到达。
(2)虚电路交换：通信前必须先建立一条"逻辑链路"（虚电路）。所有分组都必须沿着这条事先建好的虚电路传输，需要虚呼叫的建立和拆除。

三、网络协议与分层模型

协议分层概念　　【核心思想】：将网络功能分层，同等层之间用"协议"通信，相邻层之间用"接口"交互。
ISO-OSI 模型： 应用层，表示层，会话层，传输层，网络层，数据链路层，物理层（7 层）
TCP/IP 模型： 应用层，传输层，网络层，网际层（4 层）

ISO-OSI 模型与 TCP/IP 模型对比

【物理层】：透明传送比特流（0 和 1）。　【数据链路层】：在相邻结点线路上无差错传送"帧"。
【网络层】：负责主机间的报文传输，选择路由，找到目的站。（核心协议：IP）【传输层】：为两个进程提供端到端的可靠服务。（核心协议：TCP、UDP）
【应用层】：确定进程通信性质，满足用户需求。（核心协议：HTTP,DNS,FTP,SMTP 等）
【模型差异】：OSI 模型包含会话层、表示层，TCP/IP 模型将它们合并到了应用层。TCP/IP 没有明确的网络接口层，通常对应 OSI 的物理层、数据链路层。
【层级归属】：IP 协议属于网络层（或 TCP/IP 的网际层）。

四、协议：IP、TCP 与 UDP

IP 协议（网络层）

【特点】：健壮（路由器故障可绕路）、平台无关（兼容各类计算机）。
【局限】：不能保证数据包的到达顺序，也不保证一定到达。

TCP 协议（传输层）

【特点】：面向连接、可靠。在 IP 之上解决了"顺序"和"丢包"问题，但代价是速度相对较慢。
【用途】：HTTP、FTP 等需要准确无误的场景。

UDP 协议（传输层）

【特点】：无连接、不可靠。不保证顺序，不保证到达。但速度极快。
【用途】：DNS、视频流、实时游戏等对速度要求高、允许少量丢包的场景。
【注意】：Java 不支持发送原始 IP 数据报，只支持 TCP 和 UDP，以及基于它们的应用层协议。

五、IP 地址与域名系统 DNS

IPv4 地址（32 位，点分十进制，如 192.168.1.1）

【分类判断】（看第一个数字）：A 类：1~126（0 和 127 保留）。网络号 8 位，主机号 24 位。B 类：128~191。网络号 16 位，主机号 16 位。
C 类：192~223。网络号 24 位，主机号 8 位。D 类：224~239（用于组播/多播）。E 类：240~255（保留实验用）。

IPv6 地址（128 位，冒分十六进制）

【格式】：8 个区块，每个区块 4 个十六进制数，用冒号隔开。如 2001:0db8:0000:0000:0000:0000:1428:57ab
【零压缩规则】：(1)每个区块前导的 0 可以省略（如 0db8 可写为 db8）。(2)连续的多个全 0 区块，可以用双冒号"::"代替。但"::"在一个地址中只能出现一次。
(3)IPv4 嵌入 IPv6：前 96 位采用冒分十六进制，后 32 位使用 IPv4 的点分十进制。两者之间必须用冒号隔开，标准写法如::FFFF:192.168.0.1，
不能使用点号去连接十六进制部分和 IPv4 部分。

DNS（域名系统）

【作用】：将域名（如 www.szu.edu.cn）转换成 IP 地址。
【解析过程】：浏览器先向 DNS 服务器请求 IP->拿到 IP 后->浏览器直接向该 IP 的 Web 服务器发 HTTP 请求。
【为何不直接用 IP 访问】：许多服务器的 HTTP 请求头中包含"Host"字段，服务器会检查 Host 是否为域名，若直接使用 IP 访问可能会被拒绝或重定向。

六、信号与数据速率计算

电平与比特的关系【公式】：若一个数字信号有 L 个电平，每个电平需要的比特数为 log2(L)。【示例】：8 个电平，每个电平需要 log2(8)=3 位。
香农定理（有噪声通道的极限）【公式】：通道容量(bps)=带宽(Hz)*log2(1+信噪比 SNR)
奈奎斯特定理（无噪声通道的极限）【公式】：比特率(bps)=2*带宽(Hz)*log2(L)，L 为电平数量。

七、端口号与常见应用

【概念】：IP 地址用来定位"哪台电脑"，端口号用来定位"电脑上的哪个程序"。端口号是 16 位整数（0~65535）。
【端口分类】：0~1023：熟知端口号（被系统核心服务占用）。　1024~49151：注册端口号。　49152~65535：临时端口号（客户端随机使用）。
【常见熟知端口】：FTP：21（控制连接），20（数据连接）　Telnet：23（远程登录）　SMTP：25（发送邮件）　DNS：53（域名解析，UDP/TCP）
HTTP：80（网页浏览）　HTTPS：443（加密网页）
【端口独立性】：这里注意，TCP 和 UDP 的端口号是相互独立的。同一个程序完全可以同时占用 TCP 的 80 端口和 UDP 的 80 端口，两者互不干扰。

八、Internet 标准组织

【IETF】：Internet 工程任务组。民间团体，跟踪而非引导。标准：TCP/IP、MIME、SMTP。文档称为 RFC（征求意见稿）。
这里注意，RFC 文档一经发布就不再改动，是查找协议具体实现细节的权威来源。
【W3C】：万维网联盟。厂商组织，缴纳会费。引导标准：HTTP、HTML、XML。

九、代码

【知识点】：Socket 数据传输的"流串链"
【原理解释】：Socket 底层传输的是"字节（byte）"，但在 Java 中处理文本时，通常需要"按行读取字符串"。
因此需要使用"过滤器流"将字节流包装成字符流，再包装成缓冲流。

读文本标准方式（从 Socket 读取数据）

【API 串链顺序】：Socket.getInputStream()->InputStreamReader(字节转字符)->BufferedReader(缓冲并按行读取)->readLine()
【完整样例代码】：

```
Socket chatSocket = new Socket("127.0.0.1", 5000);                              //1.建立 Socket 连接
InputStreamReader stream = new InputStreamReader(chatSocket.getInputStream());  //2.把字节流转换成字符流（解决编码问题）
BufferedReader reader = new BufferedReader(stream);                             //3.包装成缓冲流，实现按行读取
String message = reader.readLine();                                             //4.读取一行数据（读到末尾返回 null）
```

写文本标准方式（向 Socket 写入数据）

【API 串链顺序】：Socket.getOutputStream()　->PrintWriter(打印流，自动处理换行)　->println()　->flush()
【完整样例代码】：

```
Socket chatSocket = new Socket("127.0.0.1", 5000);                  //1.建立 Socket 连接
PrintWriter writer = new PrintWriter(chatSocket.getOutputStream());  //2.获取输出流并包装成打印流
writer.println("message to send");                                   //3.写入数据
writer.flush();                                                      //4.这里注意，写入完成后必须调用 flush()，否则数据可能留在缓冲区未发送
```

十、补丁-网络地址转换（NAT）

NAT 的概念
【理解】：NAT（Network Address Translation）是一种网络协议转换技术，用于在不同网络（如私有局域网与公共互联网）之间转换 IP 地址。
NAT 的主要作用（简答/综合题高频考点）
(1) 解决 IPv4 地址短缺：允许内部私有网络使用不可路由的私有 IP 地址，通过 NAT 设备将其转换为合法的公共 IP 地址，从而访问互联网。
(2) 网络隔离与安全性：隐藏内部网络拓扑结构，外部网络无法直接通过公共 IP 定位到内部的具体私有设备，提供了一层天然的安全屏障。
(3) 连接共享：允许多个内部设备共享同一个或少数几个公共 IP 地址。NAT 设备通过映射不同的"端口号"来区分和转发来自不同内部设备的连接请求。

第二章：I/O STREAM（输入输出流）

一、核心概念

流（Stream）的概念
【通俗理解】：流就像是一根水管，数据就是水管里的水。程序通过"输入流"从外部（源）把水抽进来，通过"输出流"把水排到外部（目的地）。
【源与目的地】：可以是文件、键盘、鼠标、内存、显示器窗口或网络。
字节流与字符流的区别
【字节流】（InputStream/OutputStream）：一次读入或输出 8 位（1 个字节）的二进制数据。适合处理所有类型的数据（如图片、音频、视频、纯文本）。
【字符流】（Reader/Writer）：一次读入或输出 16 位（2 个字节）的字符数据。专门用于处理文本数据，内部包含了字符编码的转换逻辑（如 UTF-8、GBK）。

二、基础操作

输出流的基本方法（OutputStream）
【write(int b)】：写入单个字节（0~255）。
【write(byte[] data)】：写入整个字节数组。
【write(byte[] data, int offset, int length)】：从 offset 开始写入 length 个字节。
【flush()】：强迫缓冲的流发送数据，即使缓冲区还没有满。
【close()】：关闭流并释放相关资源。
输入流的基本方法（InputStream）
【read()】：无参数。从源中读取 1 字节数据，作为一个 0 到 255 的 int 返回。如果流结束，返回-1。这里注意，read()会阻塞等待，直到有数据可读。
【read(byte[] input)】：读取多个字节填充数组，返回实际读取的字节数。
【skip(long n)】：跳过 n 个字节。
【available()】：返回当前流中不阻塞可读取的字节数。
有符号与无符号数的转换（易错点）
【原理】：Java 中的 byte 是有符号的（-128 到 127），而 read()返回的是 0-255 的无符号 int。
【转换公式】：当把 read()的返回值强转为 byte 时，如果原值大于 127，会变成负数。若要还原为正常的 0-255 整数，需使用：int i= b >=0 ? b : 256 + b;

三、过滤器与串链机制

过滤器流的概念
【作用】：节点流（如 FileInputStream）只能进行基本的字节读写。过滤器流套在节点流外面用于提供额外的功能，如缓冲、数据格式转换、编码转换等。
过滤器串链规则（填空/编程核心）
【连接方式】：过滤器通过其构造函数与底层流连接。例如：BufferedInputStream bin = new BufferedInputStream(fin);
【这里注意】：串链完成后，应该只使用链中最后一个（最外层）的过滤器进行实际的读写。
混合调用同一个源的底层流和高层流的 read/write 方法，会违反过滤器流的隐含约定，导致数据错乱。
常用过滤器流分类
【缓冲流】：BufferedInputStream / BufferedOutputStream / BufferedReader / BufferedWriter。在内存中开辟缓冲区，减少与硬盘/网络的物理交互次数，
大幅提升效率。默认缓冲区大小：输入区 2048 字节，输出区 512 字节。
【数据流】：DataInputStream / DataOutputStream。允许直接读写 Java 基本数据类型（如 int, float, long），而不需要手动转换为字节。
【打印流】：PrintStream / PrintWriter。提供 println()、print()等便捷方法。PrintStream 可设置 autoFlush=true，在每次换行或写入字节数组时自动刷新。
【转换流】：InputStreamReader / OutputStreamWriter。字节流与字符流之间的桥梁，可以在构造时指定字符编码（如 UTF-8）。
【数组流】：ByteArrayInputStream / ByteArrayOutputStream / CharArrayReader / CharArrayWriter。以内存中的字节数组或字符数组作为源或目的地，
不涉及外部文件。

四、猜测的考点

flush()与 close()的时机
【这里注意】：在写入数据完成后，调用 flush()非常重要，它可以强迫缓冲的流发送数据。关闭流（close()）之前，必须确保数据已经刷新，否则缓冲区中
未满的数据会丢失。长时间运行且未关闭流的应用，会导致文件句柄或网络端口资源泄露。
多字节发送与读取的效率
【这里注意】：TCP/IP 协议会缓存数据。使用 write(byte[])或 read(byte[])进行批量传送，效率远高于单个字节的 write(int)或 read()。
中文乱码与编码转换（经典编程场景）
【场景】：如果直接从 HTTP 流中提取中文数据，可能会出现"?????"乱码。
【解决思路】：HTTP 默认传输编码通常是 ISO-8859-1。需要先用 ISO-8859-1 将读到的字符还原为原始字节数组，然后再用正确的编码（如 GB2312 或 UTF-8）
重新构建字符串。
【代码体现】：byte[] b = utf8_value.getBytes("ISO-8859-1"); String name = new String(b, "GB2312");
DataOutputStream 的字节序
【这里注意】：DataOutputStream 类提供了多字节（如 4 字节的 int、8 字节的 long）的写入方法，并且是以 big-endian（大端）格式写入的。

五、代码

文件读写基础格式

```
FileInputStream fin = new FileInputStream("data.txt");        FileOutputStream fout = new FileOutputStream("out.txt");
FileReader fr = new FileReader("text.txt");                   FileWriter fw = new FileWriter("out.txt");
```

缓冲流与按行读取格式

```
BufferedReader br = new BufferedReader(new FileReader("input.txt"));       BufferedWriter bw = new BufferedWriter(new FileWriter("output.txt"));
String line;          while((line = br.readLine()) != null) {bw.write(line);    bw.newLine(); // 写入换行符}bw.flush(); // 这里注意，关闭前必须 flush
```

数据流读写格式

```
DataOutputStream dos = new DataOutputStream(new FileOutputStream("data.dat"));      dos.writeInt(100);          dos.writeChars("Hello");
DataInputStream dis = new DataInputStream(new FileInputStream("data.dat"));         int num = dis.readInt();
```

字节数组流格式（内存操作）

```
ByteArrayOutputStream baos = new ByteArrayOutputStream();                            baos.write('A');
ByteArrayInputStream bais = new ByteArrayInputStream(baos.toByteArray());
```

转换流与指定编码格式

```
InputStreamReader isr = new InputStreamReader(socket.getInputStream(), "UTF-8");
OutputStreamWriter osw = new OutputStreamWriter(socket.getOutputStream(), "GB2312");
```

六、样例代码与解析

文件字符流读写与缓冲流综合应用【场景】：读取 input.txt，在每行前面加上行号，写入 output.txt。

```
    import java.io.*;     public class FileBufferExample {     public static void main(String args[]) {
        try{   FileReader fr = new FileReader("input.txt");            BufferedReader input = new BufferedReader(fr);
               FileWriter fw = new FileWriter("output.txt");            BufferedWriter output = new BufferedWriter(fw);
               String s = null;            int i = 0;
               while ((s = input.readLine()) != null) {i++;   output.write(i + ": " + s);   output.newLine();}
               output.flush();      // 这里注意：关闭流之前必须刷新，否则数据可能丢失
               output.close();          input.close();          fw.close();          fr.close();          }
        catch (IOException e) {              System.out.println(e);          }     }          }
```

文件字节流与字符串转换【场景】：将中文字符串以字节形式写入文件，再分块读出并转换为字符串。

```
    import java.io.*;     public class FileByteExample {     public static void main(String args[]) {
        File file = new File("hello.txt");          byte b[] = "深圳大学".getBytes(); // 字符串转字节数组
        try { FileOutputStream output = new FileOutputStream(file);          output.write(b);          output.close();
              FileInputStream input = new FileInputStream(file);             int n = 0;
              // 下面循环每次最多读 2 个字节，循环内将读取的有效字节转换为字符串
              while ((n = input.read(b, 0, 2)) != -1) { String str = new String(b, 0, n);      System.out.println(str); }      input.close();      }
        catch (IOException e) {              System.out.println(e);          }     }     }
```

编码格式转换编程（应对 HTTP 流中文乱码）【场景】：从 HTTP 流中提取 UTF-8 编码的中文参数，但环境默认是 ISO-8859-1，需要转换为 GB2312。

```
    public class EncodingConversion {     public static void main(String[] args) {          // 假设这是从 HTTP 流中直接读取到的被错误解码的字符串
        String utf8_value = "æµ   è¯ ";          byte[] b = utf8_value.getBytes("ISO-8859-1");     // 1. 先用 ISO-8859-1 将其还原为原始的字节数组
        try { String name = new String(b, "GB2312");   System.out.println("转换后的中文: " + name); }//2.再用正确的编码（GB2312）重新构建字符串
        catch (UnsupportedEncodingException e) {              e.printStackTrace();          }     }   }
```

Socket 数据传输的串链（第一章+第二章）【场景】：从 Socket 读取文本数据，以及向 Socket 写入文本数据。这是后续所有 Socket 编程题的底层骨架。

```
    import java.net.*;     import java.io.*;               public class SocketStreamChain {     public static void main(String[] args) {
        try {          Socket chatSocket = new Socket("127.0.0.1", 5000);
            // 【读数据】：字节流 -> 转换流(指定编码) -> 缓冲流(按行读)
            InputStreamReader streamIn = new InputStreamReader(chatSocket.getInputStream(), "UTF-8");
            BufferedReader reader = new BufferedReader(streamIn);          String message = reader.readLine(); // 读到末尾返回 null
            // 【写数据】：字节流 -> 转换流 -> 打印流(自动处理换行)
            OutputStreamWriter streamOut = new OutputStreamWriter(chatSocket.getOutputStream(), "UTF-8");
            PrintWriter writer = new PrintWriter(streamOut, true); // true 表示 autoFlush          writer.println("message to send");
            // 如果上面 PrintWriter 没加 true，这里必须手动调用 writer.flush();
            reader.close();          writer.close();          chatSocket.close();          }
        catch (IOException e) {          e.printStackTrace();          }     }     }
```

第三章：多线程与线程池（THREAD）

一、核心概念

进程与线程的区别

【进程】：具有一定独立功能的程序关于某个数据集合上的一次运行活动。它是系统进行资源分配和调度的独立单元。进程拥有独立的内存单元。
【线程】：进程的一个实体，是 CPU 调度和分派的基本单位。线程自己基本上不拥有系统资源（只有一点必不可少的寄存器和栈），但可与同属一个进程的
其他线程共享进程所拥有的全部资源。
【对比】：创建进程的开销大，创建线程的开销较小。一个程序至少有一个进程，一个进程至少有一个线程（主线程）。

线程的生命周期（5 种状态）

【新建】：线程对象被创建，但尚未调用 start()方法。
【就绪】：调用了 start()方法，线程进入线程队列排队等待 CPU 资源。
【运行】：获得 CPU 资源，开始执行 run()方法中的代码。
【阻塞/中断】：因等待 I/O、调用 sleep()、等待同步锁或调用 wait()等原因，暂时让出 CPU 并暂停执行。
【死亡】：run()方法执行完毕，或被强制终止（如 stop()），线程释放资源。

二、核心考点

线程的创建与启动

【创建方式 1】：继承 Thread 类，重写 run()方法。
【创建方式 2】：实现 Runnable 接口，实现 run()方法，然后将该实例作为参数传递给 Thread 类的构造函数。
【这里注意】：启动线程必须调用 start()方法，它会自动调用 run()方法。直接调用 run()方法只是普通的方法调用，不会开启新线程。

线程的优先级与调度

【优先级设置】：使用 setPriority(int n)方法，范围是 1 到 10（Thread.MIN_PRIORITY 到 Thread.MAX_PRIORITY），默认是 5（Thread.NORM_PRIORITY）。
【这里注意】：优先级高只代表获得 CPU 时间的概率更大，并不代表绝对的执行顺序。不同操作系统对优先级的支持可能不同（如有的系统只识别 1/5/10）。

线程同步与死锁

【同步机制】：使用 synchronized 关键字修饰代码块或方法。同一个对象上的同步代码串行执行，不同对象上的同步代码可并行。
【死锁】：两个或多个线程互相持有对方需要的锁，导致无限期等待。
【防止死锁的原则】：避免不必要的同步；同步块尽可能小；尽可能不一次同步多个对象。
【同步的替代方案】：如果不想用 synchronized，可以使用局部变量、创建不可修改的对象（如 String）、或使用 java.util.concurrent.atomic 包下的类（如
AtomicInteger）。

线程池与 Executor 框架

【作用】：限制系统中同时执行的线程数量，降低线程创建和销毁的开销，避免资源耗尽。
【核心接口与类】：
Executor：顶层接口，定义 execute()方法。
ExecutorService：子接口，提供 submit()、shutdown()等任务管理方法。
ThreadPoolExecutor：ExecutorService 的具体实现类。
Executors：工厂类，提供 newFixedThreadPool()等静态工厂方法创建线程池。
【这里注意】：使用线程池编程框架时，如果计算任务需要返回值，定义任务的类应实现 Callable 接口（而不是 Runnable），并通过 Future.get()获取结果。

线程间的协作机制（wait / notify / notifyAll）

【前提】：这三个方法均属于 Object 类的 final 方法，这意味着所有 Java 对象都可以调用它们。这里注意，它们必须在同步块或同步方法中调用，
否则会抛出 IllegalMonitorStateException 异常。
【机制说明】：
(1) wait()：当线程发现某个条件不满足时（如售票员发现没有零钱找），调用此方法。线程会进入等待状态，并释放当前持有的对象锁，直到其他线程
调用该对象的 notify() 或 notifyAll()。
(2) notify()：随机唤醒一个在该对象上等待的线程，使其重新进入就绪队列排队等待锁。
(3) notifyAll()：唤醒所有在该对象上等待的线程。通常推荐使用 notifyAll()，以避免因随机唤醒导致某些线程永远得不到执行（线程饥饿）的情况。
【应用场景】："生产者与消费者"模型，或课件中的"售票员找零"问题。当条件不满足时等待并释放锁，当其他线程改变了条件（如收到了新的大钞需要
找零）后，调用 notifyAll() 唤醒等待的线程继续执行。

三、代码

线程创建与启动

```
【继承 Thread 类】class MyThread extends Thread { public void run() { /* 线程任务 */ } }       Thread t = new MyThread();       t.start();
【实现 Runnable 接口】class MyRunnable implements Runnable { public void run() { /* 线程任务 */ } } Thread t = new Thread(new MyRunnable()); t.start();
```

线程常用控制方法

【sleep(long millis)】：让当前线程休眠指定毫秒数，进入阻塞状态，让出 CPU。
【join() / join(long millis)】：当前线程等待调用该方法的线程执行完毕（或等待指定时间）后再继续执行。
【interrupt()】：中断处于 sleep()、wait()或 join()状态的线程，使其抛出 InterruptedException 异常并提前结束阻塞。
【yield()】：让当前线程主动让出 CPU 使用权，重新进入就绪队列排队。
【setDaemon(boolean on)】：将线程设置为守护线程。当所有用户线程结束时，守护线程会自动结束。必须在 start()前调用。

线程池与 Callable/Future

```
【创建线程池】            ExecutorService pool = Executors.newFixedThreadPool(4);
【提交 Runnable 任务】    pool.submit(new MyRunnable());
【提交 Callable 任务并获取返回值】class MyCallable implements Callable<Integer> {public Integer call() { return 100; }}
        Future<Integer> future = pool.submit(new MyCallable());          int result = future.get(); // 阻塞等待结果
【关闭线程池】            pool.shutdown(); // 停止接收新任务，等待已提交任务执行完毕
```

四、样例代码与解析

多线程日志写入（同步块应用）【场景】：多个线程同时向同一个日志文件写入数据，需要避免内容交错混乱。

```
    import java.io.*;       import java.text.SimpleDateFormat;          import java.util.Date;          public class LogFile {
        // 定义一个专用的锁对象
        private final Object lock = new Object();      private final SimpleDateFormat dateFormat = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
        private final PrintWriter writer;
        public LogFile(String filename) throws IOException {          FileWriter fw = new FileWriter(filename, true);          writer = new PrintWriter(fw);     }
        public void writeLog(String message) {          // 使用同步块确保同一时刻只有一个线程能写入
            synchronized (lock) {   String timestamp = dateFormat.format(new Date());   writer.println(timestamp + " - " + message);   writer.flush();   }   }
        public void close() {          synchronized (lock) {          writer.close();          }     }}
```

使用线程池实现并发文件压缩（线程池与 Runnable 综合）【场景】：重点？将目录下的多个文件并发压缩，使用线程池控制并发数。

```
    import java.io.*;     import java.util.concurrent.*;     import java.util.zip.*;
    public class GZipAllFiles {     public static final int THREAD_COUNT = 4; // 线程池大小
        public static void main(String[] args) {
            ExecutorService pool = Executors.newFixedThreadPool(THREAD_COUNT);
            for (String filename : args) {     Runnable task = new GZipRunnable(filename);      pool.submit(task); // 提交任务给线程池    }
            pool.shutdown(); // 关闭线程池，不再接收新任务       }          }
    class GZipRunnable implements Runnable {     private String filename;     public GZipRunnable(String filename) {this.filename=filename;}     @Override
        public void run() {
            try { File input = new File(filename);                    File output = new File(filename + ".gz");
                FileInputStream in = new FileInputStream(input);                FileOutputStream out = new FileOutputStream(output);
                ZipOutputStream zipOut = new ZipOutputStream(out);     ZipEntry entry = new ZipEntry(input.getName());    zipOut.putNextEntry(entry);
                byte[] buffer = new byte[1024];               int length;
                while ((length = in.read(buffer)) != -1) {               zipOut.write(buffer, 0, length);               }
                zipOut.close();               out.close();               in.close();               }
            catch (IOException e) {          e.printStackTrace();          }     }          }
```

使用 Callable 和 Future 获取线程计算结果【场景】：在一个大数组中查找最大值，将数组分块交给多个线程计算，最后汇总结果。

```
    import java.util.concurrent.*;          public class FindMaxValue {     public static void main(String[] args) throws Exception {
            int[] array = new int[10000];          // 假设数组已填充随机数...
            ExecutorService pool = Executors.newFixedThreadPool(4);          Future<Integer>[] futures = new Future[4];
            int chunkSize = array.length / 4;
            for (int i = 0; i < 4; i++) {          int start = i * chunkSize;          int end = (i == 3) ? array.length : (i + 1) * chunkSize;
                // 提交 Callable 任务          futures[i] = pool.submit(new FindMaxTask(array, start, end));          }
            int globalMax = Integer.MIN_VALUE;
            for (Future<Integer> f : futures) {          // get()方法会阻塞，直到该线程计算完成并返回结果          int localMax = f.get();
                if (localMax > globalMax) { globalMax = localMax; }          }
            System.out.println("全局最大值: " + globalMax);          pool.shutdown();     }          }
    class FindMaxTask implements Callable<Integer> {     private int[] array;     private int start, end;
        public FindMaxTask(int[] array, int start, int end) {     this.array = array;     this.start = start;     this.end = end;     }     @Override
        public Integer call() { int max = Integer.MIN_VALUE;   for (int i = start; i < end; i++) { if (array[i] > max) max = array[i]; } return max;     }}
```

五、综合题猜测

从线程返回信息的几种方式

【问题】run()和 start()方法没有返回值，如何获取子线程的计算结果？
【解答】：(1) 轮询（Polling）：设置一个标志变量，主线程不断查询该标志，直到子线程修改标志表示完成。
(2) 回调（Callback）：子线程执行完毕后，主动调用主类（或传入的接口实例）中的某个方法来传递结果。
(3) 使用 Callable 与 Future：实现 Callable 接口的 call()方法，通过线程池的 submit()提交，返回 Future 对象，主线程通过 Future.get()阻塞获取结果。

线程池的类结构与工厂模式（简答题常客）

【问题】Java 实现线程池使用了工厂模式，什么是工厂模式？请简述线程池相关类的结构。
【解答】：
【工厂模式】：一种创建型设计模式，将对象的创建逻辑封装在独立的工厂类中，客户端无需知道具体对象的创建过程，只需通过工厂类获取对象。
这样可以解耦创建与使用，提高代码可维护性。
【类结构关系】：
Executor：顶层接口，定义 execute(Runnable)方法。
ExecutorService：子接口，扩展了 submit()、shutdown()等任务生命周期管理方法。
AbstractExecutorService：抽象实现类，提供默认实现。
ThreadPoolExecutor：核心实现类，提供线程池的具体功能（如创建、管理、调度）。
ScheduledExecutorService：子接口，支持定时和周期性任务执行。
Executors：工厂类，提供 newFixedThreadPool()等静态方法创建线程池实例。

第四章：Internet 地址（InetAddress）

一、核心概念

IP 地址与域名的关系

【IP 地址】：连接到 Internet 的节点或主机的唯一数字标识（如 IPv4 的 32 位数字）。
【域名】：为了方便人类记忆而发明的字符标识（如 www.szu.edu.cn）。
【DNS 服务】：域名系统（Domain Name System），负责将域名转换为 IP 地址。
【对应关系】：
(1) 一台主机可能有多个 IP 地址。
(2) 一个 IP 地址可能被多个域名指向。
(3) 一个域名可能指向多个 IP 地址（DNS 会从中随机选择一个进行访问）。

为什么不直接通过 IP 访问网站？

【现象】：在浏览器输入域名可以正常访问，但输入对应的 IP 地址可能会被拒绝或重定向。
【原因】：服务端对此做了限制。在 HTTP 协议的请求头中有一个"Host"字段。通过域名访问时，Host 的值是域名；通过 IP 访问时，Host 的值是 IP。
许多服务器（如深大、126 邮箱）会检测 Host 字段，若发现是 IP 直接访问，可能会拒绝请求或将其重定向到相应的域名上。

二、IP 地址格式与规范：**见第一章**

三、InetAddress 类机制

创建 InetAddress 对象

【前提】：InetAddress 类没有 public 的构造方法，必须依赖其静态方法来创建对象。
【四种创建方法及区别】：
(1) getLocalHost()：获取本机的 InetAddress 对象。这里注意，当本机绑定了多个 IP 地址时，只返回第一个地址。
(2) getByName(String host)：传入域名或 IP 地址。如果传入域名，会通过 DNS 解析获取 IP；如果传入 IP 地址，无论该 IP 是否存在，都会直接返回该 IP
（不验证 IP 正确性）。若域名不存在，抛出 UnknownHostException。
(3) getAllByName(String host)：传入域名，从 DNS 获取该域名对应的所有 IP，返回一个 InetAddress 数组。
(4) getByAddress(byte[] addr) 或 getByAddress(String host, byte[] addr)：必须通过 byte 数组形式的 IP 地址创建。不验证 IP 是否存在。
第二个重载中的 host 仅作为别名，不会触发 DNS 查询。

获取域名与 IP 的方法

【getHostName()】：返回主机名。
【getCanonicalHostName()】：查询 DNS，返回规范化的主机名（完整域名）和 IP。
【getHostAddress()】：返回字符串类型的 IP 地址。
【getAddress()】：返回 byte 数组类型的 IP 地址。这里注意，Java 的 byte 是有符号的（-128~127），如果 IP 某段大于 127，返回的 byte 会是负数，显示时
需转换为 int（若小于 0 则加 256）。

getHostName() 的 DNS 触发机制

【注意】：三种创建方式下，调用 getHostName() 的行为不同。
(1) 使用 getLocalHost() 创建：直接返回本机名，不访问 DNS。
(2) 使用域名创建（getByName("域名")）：系统已记住该域名，直接返回域名，不访问 DNS。
(3) 使用 IP 地址创建（getByName("IP") 或 getByAddress()）：这里注意，此时必须通过 DNS 服务器进行反向查找来获取域名。如果 IP 不存在或 DNS
不允许映射，则直接返回该 IP 地址。

DNS 缓存与程序安全性

【DNS 缓存】：DNS 查询开销较大，InetAddress 会缓存查找结果（默认 30 秒，可通过安全属性修改）。本机、本地 DNS 也会缓存，Java 无法控制外部缓存。
修改域名指向可能需要几个小时才能生效。
【安全性问题】：根据主机名创建 InetAddress 对象需要 DNS 查找，任意的 DNS 查找会打开隐藏通道，可能造成信息泄露。在程序中常需先测试主机是否
支持 DNS 解析，可使用 SecurityManager 的 checkConnect(String hostname, int port) 方法（port 传-1）进行检查。

四、代码

创建对象

```
InetAddress addr1 = InetAddress.getLocalHost();                          InetAddress addr2 = InetAddress.getByName("www.szu.edu.cn");
InetAddress[] addrs = InetAddress.getAllByName("www.szu.edu.cn")         InetAddress addr3 = InetAddress.getByName("192.168.1.1");//不验证 IP 是否存在
```

获取信息

```
String hostName=addr.getHostName();//获取域名     String ipStr=addr.getHostAddress();//获取字符串 IP     byte[] ipBytes=addr.getAddress();//获取数组 IP
```

网络接口获取

```
Enumeration<NetworkInterface> interfaces = NetworkInterface.getNetworkInterfaces();
```

五、样例代码与解析

获取并打印 IP 地址的字节形式（处理有符号 byte 转换）【场景】：使用 getAddress() 获取 IP 字节数组，并正确打印出无符号的十进制 IP。

```
    import java.net.*;          public class GetIPExample {     public static void main(String[] args) throws Exception {
            InetAddress address = InetAddress.getByName("www.csdn.net");          byte[] ip = address.getAddress();      System.out.print("原始 byte 输出: ");
            for (byte ipSegment : ip) {   System.out.print(ipSegment + " "); }      System.out.println();      System.out.print("转换后正确输出: ");
            for (byte ipSegment : ip) {   // 这里注意：如果 byte 大于 127，在 Java 中会显示为负数，需要加 256 还原
                int newIPSegment = (ipSegment < 0) ? 256 + ipSegment : ipSegment;
                System.out.print(newIPSegment + " ");          }          }          }
```

验证不同创建方式下 getHostName() 的 DNS 解析耗时【场景】：验证使用域名创建和使用 IP 创建时，调用 getHostName() 是否会触发 DNS 查询。

```
    import java.net.*;          public class DNSResolveTest {
        public static void main(String[] args) throws Exception {          // 1. 使用域名创建
            InetAddress address1 = InetAddress.getByName("www.szu.edu.cn");          long time1 = System.currentTimeMillis();
            System.out.println("域名创建，获取主机名:"+address1.getHostName());   System.out.println("耗时:"+(System.currentTimeMillis()-time1)+"毫秒");
            // 耗时通常为 0，因为未访问 DNS                    // 2. 使用 IP 地址创建
            InetAddress address2 = InetAddress.getByName("210.39.3.164");          long time2 = System.currentTimeMillis();
            System.out.println("IP 创建，获取主机名: " + address2.getHostName());   System.out.println("耗时: " + (System.currentTimeMillis() - time2) + " 毫秒");
            // 耗时较长，因为触发了 DNS 反向解析          }     }
```

综合应用：如何快速判断垃圾邮件发送者？【场景】：利用 DNS 黑名单机制（如 spamhaus.org）判断某个 IP 是否为已知的垃圾邮件发送者。
【思路】：将发送者的 IP 地址反转，拼接上黑名单域名，然后使用 getByName() 进行 DNS 查询。如果查询成功（未抛出异常），说明该 IP 在黑名单中。

```
    import java.net.*;
    public class SpamCheck {      // 定义黑名单域名后缀
        public static final String BLACKHOLE = "sbl.spamhaus.org";
        public static void main(String[] args) {
            String testIP = "192.168.1.1"; // 这里可以替换为要测试的 IP
            if (isSpammer(testIP)) {          System.out.println(testIP + " is a known spammer.");          }
            else {          System.out.println(testIP + " appears legitimate.");          }     }
        private static boolean isSpammer(String arg) {
            try { InetAddress address = InetAddress.getByName(arg);          byte[] quad = address.getAddress();          String query = BLACKHOLE;
                // 将 IP 地址的四个字节反转并拼接
                for (byte octet : quad) {   int unsignedByte = octet < 0 ? octet + 256 : octet; query = unsignedByte + "." + query; }
                // 尝试解析拼接后的黑名单域名
                InetAddress.getByName(query);          return true; // 解析成功，说明在黑名单中
            }
            catch (UnknownHostException e) {          return false; // 解析失败，说明不在黑名单中          }     }     }
```

第五章：URL 与 URI

一、核心概念

URL 与 URI 的定义
【URL】：统一资源定位符（Uniform Resource Locator）。不仅标识资源，还明确指出如何访问该资源（包含协议、主机、端口等网络定位信息）。
【URI】：统一资源标识符（Uniform Resource Identifier）。范围更广，包含 URL 和 URN（统一资源名称）。
主要作用是按照 RFC 3986 标准规范地标识和解析资源字符串，不直接提供网络连接功能。
【关系】：所有 URL 都是 URI，但并非所有 URI 都是 URL。

二、URL 结构与解析方法

URL 标准格式

【结构】：protocol://host:port/path?query#ref
【对应 Java 字段】：protocol -> 协议（如 http, ftp, file, jar）　host -> 主机名或 IP 地址
port -> 端口号（若未显式指定，默认 http 为 80，https 为 443）　path -> 资源路径
query -> 查询参数（?后面的键值对）　ref -> 锚点或片段（#后面的部分）

解析方法

【getProtocol()】：返回协议字符串。　【getHost()】：返回主机名或 IP。
【getPort()】：返回端口号。这里注意，若 URL 中未显式指定端口，该方法返回 -1，而不是默认端口号（如 80）。
【getPath()】：返回路径部分，不含查询参数和锚点。　【getQuery()】：返回查询字符串（?后面的部分，不含?）。
【getFile()】：返回路径加查询字符串（即 getPath() + "?" + getQuery()，若无查询则等同于 getPath()）。
【getRef()】：返回锚点部分（#后面的部分，不含#）。　【toExternalForm()】或【toString()】：返回完整的 URL 字符串表示。

三、URI 与 URL 的核心区别

功能定位差异

【URL】：可调用 openConnection() 方法建立网络连接，获取数据流。
【URI】：仅提供标识和解析功能，无法直接打开网络连接。若需访问资源，需先通过 uri.toURL() 转换（可能抛异常）。

协议支持范围

【URL】：仅支持 Java 运行环境中已注册协议处理器（Handler）的协议。若传入未知协议（如 myapp://），会抛出 MalformedURLException。
【URI】：支持任何符合语法的字符串，不验证协议是否可用。

字符编码与转义

【URI】：构造时会自动处理保留字符的编码（如空格转为%20，中文转为%XX 形式）。
【URL】：对非法字符较敏感，部分构造方法遇到空格或中文可能直接抛出异常，需提前使用 URLEncoder 处理。

equals() 方法的陷阱

【URL.equals()】：不仅比较字符串，还会触发 DNS 解析，比较两个主机名是否指向同一个 IP。若 DNS 不可用或主机名无法解析会导致程序阻塞或抛出异常。
【URI.equals()】：仅进行严格的字符串逐字比较，不会触发网络操作，更安全高效。

四、URL 构造函数与相对路径解析

常用构造函数

【URL(String spec)】：通过完整 URL 字符串创建。　【URL(String protocol, String host, int port, String file)】：分段传入参数创建。
【URL(URL context, String spec)】：基于基准 URL（context）解析相对路径（spec）创建新 URL。

相对路径解析规则（与 HTML 规则一致）

【情况 1】：spec 以 / 开头 -> 替换基准 URL 的完整路径，仅保留协议、主机和端口。
【情况 2】：spec 以 ./ 开头或不以 / 开头 -> 基于基准 URL 路径的最后一个 / 进行拼接。
【情况 3】：spec 以 ../ 开头 -> 向上回退一级目录后再拼接。
【示例】：基准 URL 为 http://example.com/a/b/c.html
相对路径 /d/e.html -> http://example.com/d/e.html　　相对路径 f/g.html -> http://example.com/a/b/f/g.html

五、代码

URL 字段提取（"//"后为返回值形式）

```
URL url = new URL("http://example.com:8080/path?query=1#ref");      String protocol = url.getProtocol(); // "http"
String host = url.getHost(); // "example.com"                       int port = url.getPort(); // 8080 (未指定时返回-1)
String path = url.getPath(); // "/path"                             String query = url.getQuery(); // "query=1"
String file = url.getFile(); // "/path?query=1"                     String ref = url.getRef(); // "ref"
```

相对路径解析

```
URL base = new URL("http://example.com/docs/");        URL relative = new URL(base, "guide.html"); // 解析为 http://example.com/docs/guide.html
```

特殊字符编码处理

【URLEncoder.encode(String s, String enc)】：将字符串编码为 application/x-www-form-urlencoded 格式（空格变+，中文变%XX）。
【URLDecoder.decode(String s, String enc)】：将编码后的字符串还原。

六、样例代码与解析

URL 各部分解析与 -1 端口处理【场景】：解析任意 URL，并正确处理未指定端口的情况。

```
    import java.net.*;     public class URLParserExample {     public static void main(String[] args) throws MalformedURLException {
            URL url = new URL("http://www.szu.edu.cn/news/detail?id=100#content");
            System.out.println("协议: " + url.getProtocol());          System.out.println("主机: " + url.getHost());
            int port = url.getPort();          // 这里注意：getPort()未指定时返回-1，需根据默认协议判断
            if (port == -1) {          String protocol = url.getProtocol();          port = protocol.equals("https") ? 443 : 80;          }
            System.out.println("端口: " + port);   System.out.println("路径: " + url.getPath());   System.out.println("查询: " + url.getQuery());
            System.out.println("完整文件路径: " + url.getFile());   System.out.println("锚点: " + url.getRef());          } }
```

URL 相对路径解析【场景】：在爬虫或网页解析中，经常需要将相对链接转换为绝对链接。

```
    import java.net.*;          public class RelativeURLResolver {     public static void main(String[] args) throws MalformedURLException {
        URL baseUrl = new URL("http://example.com/projects/java/");
        // 情况 1：以 / 开头，替换整个路径
        URL abs1 = new URL(baseUrl, "/api/v1/users");          System.out.println("绝对路径 1: " + abs1);// 输出: http://example.com/api/v1/users
        // 情况 2：不以 / 开头，追加到当前目录
        URL abs2 = new URL(baseUrl, "guide.pdf");          System.out.println("绝对路径 2: " + abs2);// 输出: http://example.com/projects/java/guide.pdf
        // 情况 3：向上级目录跳转
        URL abs3=new URL(baseUrl, "../python/readme.md");System.out.println("绝对路径 3:"+abs3);//输出:http://example.com/projects/python/readme.md}}
```

URL 中查询参数的编码与解码【场景】：构造 HTTP GET 请求时，查询参数中的中文或空格必须经过编码，否则会导致 MalformedURLException。

```
    import java.net.*;          import java.io.*;          public class QueryEncodingExample {     public static void main(String[] args) {
            try { // 原始参数包含中文和空格
                String rawQuery = "name=张三 & city=深圳";
                // 使用 UTF-8 编码
                String encodedQuery = URLEncoder.encode(rawQuery, "UTF-8");          System.out.println("编码后: " + encodedQuery);
                // 输出: name%3D%E5%BC%A0%E4%B8%89+%26+city%3D%E6%B7%B1%E5%9C%B3
                // 解码还原
                String decodedQuery = URLDecoder.decode(encodedQuery, "UTF-8");          System.out.println("解码后: " + decodedQuery);
                // 输出: name=张三 & city=深圳
                // 构造安全 URL
                String safeUrl = "http://api.example.com/search?q=" + encodedQuery;     URL url = new URL(safeUrl);   System.out.println("安全 URL:"+url);}
            catch (Exception e) {          e.printStackTrace();          }     }          }
```

七、URI 核心 API 与代码实战

URI 核心 API 与代码

URI 的构造

【构造方法】：URI(String uri) 或 URI(String scheme, String schemeSpecificPart, String fragment)
【这里注意】：构造 URI 时只检查语法，不检查协议是否存在（即使协议是瞎编的，只要语法对就不会报错）。

```
URI voice = new URI("tel:+1-800-9988-9938"); // 合法，不检查 tel 协议      URI web = new URI("http://www.a.com/a.jsp#id=hbc");
```

URI 各部分的获取

【核心方法】：getScheme()：获取协议（如 "http"）。　getSchemeSpecificPart()：获取解码后的协议特定部分。
getRawSchemeSpecificPart()：获取未解码的协议特定部分。　getFragment()：获取解码后的片段（#后面的部分）。
isOpaque()：判断是否为不透明 URI（不透明 URI 不能获取 host、port 等信息）。
相对 URI 的解析与转换： resolve(URI other)：将相对 URI 解析为绝对 URI。　relativize(URI uri)：将绝对 URI 转换为相对 URI。

经典样例代码与实战解析

URI 的构造与相对路径解析（resolve）【场景】：在爬虫或网页解析中，将页面中的相对链接转换为绝对链接。

```
    import java.net.*;          public class URIResolveExample {     public static void main(String[] args) throws URISyntaxException {          // 基准绝对 URI
            URI absolute = new URI("http://www.example.com/docs/");
            // 情况 1：相对路径
            URI relative1 = new URI("guide.html");          URI resolved1 = absolute.resolve(relative1);          System.out.println("解析结果 1: " + resolved1);
            // 输出: http://www.example.com/docs/guide.html
            // 情况 2：向上级目录跳转的相对路径
            URI relative2 = new URI("../images/logo.png");          URI resolved2 = absolute.resolve(relative2);     System.out.println("解析结果 2: " + resolved2);
            // 输出: http://www.example.com/images/logo.png          }     }
```

绝对 URI 转相对 URI（relativize）【场景】：生成网页中的相对超链接。

```
    import java.net.*;          public class URIRelativizeExample {     public static void main(String[] args) throws URISyntaxException {
            URI base = new URI("http://www.example.com/");          URI target = new URI("http://www.example.com/images/logo.png");
            // 将绝对 URI 转换为相对于 base 的相对 URI
            URI relative = base.relativize(target);          System.out.println("相对 URI: " + relative);          // 输出: images/logo.png     }     }
```

URLEncoder 的"盲目编码"陷阱及正确处理

【场景】：构造 HTTP GET 请求的查询字符串时，如果直接对整个 URL 使用 URLEncoder.encode()，会导致协议头（如 http://）和分隔符（如?、&）也被错误编码。
【错误示范】：URLEncoder.encode("http://www.a.com/search?q=测试", "UTF-8") 会变成 http%3A%2F%2F...，导致 URL 失效。
【正确思路】：必须将需要编码的"值"单独编码，然后再与未编码的 URL 结构拼接。

```
    import java.net.*;          public class URLEncoderTrap {     public static void main(String[] args) {
            String baseUrl = "http://www.example.com/search?";          String key1 = "q";          String value1 = "Java 编程"; // 包含空格和中文
            String key2 = "sort";          String value2 = "time";
            try {// 【正确做法】：只对键值对的值进行编码，然后手动拼接 URL 结构
                String encodedValue1 = URLEncoder.encode(value1, "UTF-8");          // 输出: Java+%E7%BC%96%E7%A8%8B (空格变成了+)
                String safeUrl = baseUrl + key1 + "=" + encodedValue1 + "&" + key2 + "=" + value2;          System.out.println("安全的 URL: " + safeUrl);
                // 输出: http://www.example.com/search?q=Java+%E7%BC%96%E7%A8%8B&sort=time          }
            catch (UnsupportedEncodingException e) {          e.printStackTrace();          }     }     }
```

第六章：HTTP 协议与 Cookie 机制

一、核心概念

HTTP 协议基础

【概念】：HTTP（超文本传输协议）是 Web 浏览器和 Web 服务器之间通信的标准协议。它定义了客户端如何向服务器请求资源，以及服务器如何返回资源。
【特性】：(1) 基于 TCP 协议：HTTP 在传输层使用 TCP 协议，保证了数据传输的可靠性。
(2) 无状态协议：服务器发送响应后，不保存浏览器的任何请求状态信息。每次请求对服务器来说都是独立的。
(3) 灵活的数据类型：可以传输 HTML、图片、音频、视频等任何可以用字节表示的数据（通过 MIME 类型标识）。

连接管理机制

【非持续连接（HTTP 1.0）】：每次请求/响应都要建立一次新的 TCP 连接，传输完毕后立即断开。缺点是建立连接的开销大，不适合包含大量对象的网页。
【持续连接（HTTP 1.1）】：服务器在发出响应后保持该 TCP 连接，后续的请求与应答都通过同一个连接传送。
非流水线：客户端必须等收到前一个响应，才能发下一个请求。　流水线：客户端不用等前一个响应，就可以连续发出多个请求（效率更高）。

会话状态保持：Cookie 与 Session

【Cookie（客户端技术）】：服务器在 HTTP 响应头中通过 Set-Cookie 字段将少量数据发给浏览器，浏览器保存在本地。后续请求该服务器时，浏览器会
在 HTTP 请求头中带上这些数据。
【Session（服务器端技术）】：服务器在内存中为每个用户的浏览器创建一个独享的 Session 对象，用于保存用户的状态数据。

二、HTTP 报文结构

客户端请求报文

【结构】：请求行+请求头（HTTP 首部） + 空行 + 请求体（可选）。
(1) 请求行：格式为 方法 资源路径 HTTP 版本（如 GET /index.html HTTP/1.1）。
(2) 请求头：格式为 关键字: 值，必须以 \r\n 结尾。　Host：指定请求的主机名（必须包含，用于虚拟主机识别）。
User-Agent：标识客户端浏览器信息。　Accept：声明客户端能处理的 MIME 类型（如 text/html, image/jpeg）。
(3) 空行：告诉服务器请求头结束，即使没有请求体，空行也不能省略。

服务器响应报文

【结构】：状态行 + 响应头（信息头） + 空行 + 响应体。
(1) 状态行：格式为 HTTP 版本 状态码 状态描述（如 HTTP/1.1 200 OK）。
(2) 响应头：描述响应体的属性。　Content-Type：响应体的 MIME 类型及字符编码（如 text/html; charset=UTF-8）。
Content-Length：响应体的字节数。　Date：服务器当前时间。

HTTP 状态码

【1xx】：请求已收到，需进一步处理（如 100 Continue）。　【2xx】：请求成功（如 200 OK）。
【3xx】：重定向（如 301 永久移动，302 临时移动，304 未修改/使用缓存）。
【4xx】：客户端错误（如 400 语法错误，401 未授权，403 禁止访问，404 资源未找到）。
【5xx】：服务器端错误（如 500 内部服务器错误，502 网关错误）。

HTTP 请求方法

【GET】：用于获取 URL 标识的资源。安全、幂等。参数附加在 URL 后，数据量小。
【POST】：用于向服务器提交数据（如表单）。不安全，数据放在请求体中，数据量大。
【HEAD】：与 GET 类似，但服务器只返回响应头，不返回响应体。常用于检查资源是否更新或获取 Content-Length。
【PUT】：用于上传或更新指定 URL 的资源。　【DELETE】：用于删除指定 URL 的资源。

三、Cookie 机制与编程实现

Cookie 的编码与属性

【编码要求】：只能是非空白符的 ASCII 文本，不能包含逗号或分号。若需保存中文或特殊字符，必须进行 URL 编码（如使用 URLEncoder）。
【作用域控制】：domain：指定 Cookie 对哪些域名有效（如 domain=.szu.edu.cn）。　path：指定 Cookie 对哪些路径有效（如 path=/home）。
【过期属性】：expires：指定过期的绝对时间（如 Wed, 21-Dec-2015 15:23:00 GMT）。
Max-Age：指定过期的相对秒数（如 Max-Age=3600 表示 1 小时后过期）。

Java 中的 CookieManager

【启用 Cookie】：在 Java 中默认不自动处理 Cookie，必须手动启用。
CookieManager manager = new CookieManager();　CookieHandler.setDefault(manager); **// 设置为全局默认**
【接收策略（CookiePolicy）】：
CookiePolicy.ACCEPT_ALL：**接受所有 Cookie。**　CookiePolicy.ACCEPT_NONE：**不接受任何 Cookie。**
CookiePolicy.ACCEPT_ORIGINAL_SERVER：**只接受第一方（原始服务器）的 Cookie。**
【自定义策略】：通过实现 CookiePolicy 接口，重写 shouldAccept(URI uri, HttpCookie cookie) 方法，可以编写自定义的过滤逻辑（如屏蔽特定域名的 Cookie）。

四、猜测考点：HTTP 缓存机制

【为什么需要缓存】：减少网络传输，加快客户端响应速度。默认情况下，GET 请求的页面可以且应该被缓存，POST/HTTPS 不应缓存。
【控制缓存的响应头字段】：　Expires：指定资源过期的绝对时间。
Cache-Control：提供更细粒度的控制（优先级高于 Expires）。　max-age=[秒数]：从现在起缓存过期的秒数。
no-store：绝对不缓存。　no-cache：可以缓存，但每次使用前必须向服务器验证（协商缓存）。
Last-Modified：资源最后一次修改的时间。　Etag：资源改变时的唯一标识符。
【缓存判断逻辑】：当本地缓存的副本时间早于 Last-Modified，或者本地缓存的 Etag 与服务器返回的不同时，才会真正向服务器发送 GET 请求获取新数据，
否则直接使用本地缓存。
【no-cache 与 no-store 的区别】：no-store：绝对不缓存。客户端和服务器都不保留任何关于请求和响应的副本，每次都必须从服务器重新下载完整资源。
no-cache：可以缓存，但每次使用前必须向服务器发送请求进行验证（即协商缓存）。服务器会通过比对 Etag 或 Last-Modified 来判断资源是否更新，
若未更新则返回 304 状态码，客户端继续使用本地缓存。
【优先级规则】：当 HTTP 响应头中同时出现 Cache-Control 和 Expires 字段时，Cache-Control 的优先级更高，其设置会覆盖 Expires 的值。

五、代码

启用并配置 Cookie 管理器

```
CookieManager manager = new CookieManager();                              // 1. 创建管理器
manager.setCookiePolicy(CookiePolicy.ACCEPT_ORIGINAL_SERVER);             // 2. 设置策略（例如只接受第一方 Cookie）
CookieHandler.setDefault(manager);                                        // 3. 注册为全局默认
```

自定义 Cookie 策略类（拦截特定域名）

```
    import java.net.*;          public class CustomCookiePolicy implements CookiePolicy {     @Override
        public boolean shouldAccept(URI uri, HttpCookie cookie) {          // 获取请求的域名和 Cookie 的域名
            String uriDomain = uri.getHost();          String cookieDomain = cookie.getDomain();
            // 这里注意：如果包含 .gov，则拒绝接受
            if (uriDomain != null && uriDomain.toLowerCase().endsWith(".gov")) {          return false;          }
            if (cookieDomain != null && cookieDomain.toLowerCase().endsWith(".gov")) {          return false;   }
            return true; // 接受其他所有域     }}
```

使用 Socket 模拟发送 HTTP GET 请求（编程骨架）

```
    import java.net.*;          import java.io.*;          public class SimpleHTTPClient {     public static void main(String[] args) {
            String host = "www.szu.edu.cn";          int port = 80;          String path = "/index.html";
            try (Socket socket = new Socket(host, port)) {          // 1. 获取输出流，拼接 HTTP 请求报文
                OutputStream out = socket.getOutputStream();
                String request="GET "+path+" HTTP/1.1\r\n"+"Host: "+host+"\r\n"+"Connection: close\r\n"+"\r\n";//这里注意：请求头结束必须有一个空行
                out.write(request.getBytes("ISO-8859-1"));          out.flush();
                // 2. 获取输入流，读取响应
                BufferedReader reader = new BufferedReader(new InputStreamReader(socket.getInputStream(), "UTF-8"));          String line;
                while ((line = reader.readLine()) != null) {          System.out.println(line);          }          }
            catch (IOException e) {          e.printStackTrace();          }     }          }
```

六、综合题相关

简述 GET 与 POST 的区别

【解答】：(1) 语义不同：GET 用于获取资源，POST 用于向服务器提交/修改资源。(2) 数据位置不同：GET 的参数附加在 URL 之后（受 URL 长度限制，通常不超过 2KB）；POST 的数据放在请求体中，理论上无大小限制。(3) 安全性不同：GET 参数暴露在 URL 中，安全性较低；POST 数据在请求体中，相对安全（但并非绝对安全，需配合 HTTPS）。(4) 缓存与幂等性：GET 请求会被浏览器主动缓存，且是幂等的（多次请求结果一致）；POST 请求不会被缓存，且非幂等。

试从程序输入输出处理数据的角度，解释从浏览器输入 URL 到看到网页内容的过程。

【解答】：(1) URL 解析：浏览器解析输入的 URL，提取出协议、主机名和路径。(2) DNS 解析：通过 DNS 服务器将主机名解析为对应的 IP 地址。(3) 建立连接：客户端与目标服务器的 IP 地址建立 TCP 连接（三次握手）。(4) 发送 HTTP 请求：浏览器构建 HTTP 请求报文（包含请求行、请求头如 Host/Accept 等），通过 TCP 连接发送给服务器。(5) 服务器处理与响应：服务器接收请求，处理业务逻辑，生成 HTTP 响应报文（包含状态行、响应头如 Content-Type 以及 HTML 响应体）返回给浏览器。(6) 浏览器渲染：浏览器接收响应，解析 HTML、CSS 和 JS，最终将网页渲染展示给用户。

第七章：URLConnection 与 HTTP 高级编程

一、核心概念

URL 与 URLConnection 的区别
【URL】：只是一个资源的"地址指针"。调用 openStream() 只能盲目地获取一个输入流，无法获取 HTTP 头信息，也无法向服务器发送数据。
【URLConnection】：是应用程序与 URL 之间的"通信链接"。它不仅能读取数据，还能访问服务器发送的所有元数据（HTTP 响应头），可以配置发给服务器的
请求参数（请求头），还能向服务器发送数据（如 POST 表单）。
【关系】：URLConnection 是一个抽象类，针对 HTTP 协议，它的具体子类是 HttpURLConnection。
URLConnection 的标准使用步骤（编程骨架）
【1】：创建 URL 对象。　【2】：调用 URL.openConnection() 获取 URLConnection 对象。
【3】：配置 URLConnection 对象（设置请求方法、请求头、是否允许输出等）。这里注意，配置必须在连接建立前完成。
【4】：读取头信息（可选）。　【5】：获得输入流（getInputStream()），并读取数据。
【6】：获得输出流（getOutputStream()），并写入数据（如果是 POST/PUT 请求）。　【7】：关闭流，并调用 disconnect() 断开连接。

二、猜测考点

头信息获取【getContentLength()】：返回响应体的字节数。这里注意，如果服务器没有发送 Content-Length 头部，该方法返回 -1，而不是 0。
【getContentType()】：返回 MIME 类型（如 text/html; charset=UTF-8）。HTTP 默认的字符编码是 ISO-8859-1。
【遍历所有头字段】：因为无法预知服务器会发送哪些头，可以使用 getHeaderFieldKey(int n) 获取第 n 个字段名，getHeaderField(int n) 获取第 n 个字段值，
通过循环实现遍历。超范围返回 null。
POST 请求的配置【默认状态】：URLConnection 默认只允许读（doInput=true），不允许写（doOutput=false）。
【修改配置】：如果要发送 POST 或 PUT 请求，必须手动调用 setDoOutput(true)。
【请求方法】：如果是 HttpURLConnection 对象，还需要调用 setRequestMethod("POST")（方法名必须全大写）。
异常与错误流的处理【场景】：当请求的 URL 不存在（返回 404）或服务器内部错误（返回 500）时，服务器通常会返回一个包含错误提示的 HTML 页面。
【坑点】：此时如果调用 getInputStream()，会直接抛出 IOException。
【正确做法】：必须先通过 getResponseCode() 判断状态码。如果状态码 >= 400，必须改用 getErrorStream() 来获取服务器返回的错误页面 HTML 内容。
缓存控制字段的优先级【Expires】：指定资源过期的绝对时间。　【Cache-Control】：提供更细粒度的控制（如 max-age, no-store, no-cache）。
【优先级规则】：当响应头中同时出现 Cache-Control 和 Expires 时，Cache-Control 的优先级更高，会覆盖 Expires 的值。

三、代码

获取特定头信息

```
String type = uc.getContentType();          // MIME 类型                   int length = uc.getContentLength();      // 字节数，无该头则返回-1
String encoding = uc.getContentEncoding();// 压缩编码（非字符编码）       long date = uc.getDate();                 // 发送时间
long expired = uc.getExpiration();          // 失效时间                   long modified = uc.getLastModified();    // 最后修改时间
```

配置请求头与 Cookie

```
// 设置请求头（必须在 connect 前或 getInputStream 前调用，两行都要）：
uc.setRequestProperty("Accept", "text/html");                 uc.setRequestProperty("Content-Type", "application/x-www-form-urlencoded");
// 手动添加 Cookie 到请求头：uc.setRequestProperty("Cookie", "username=elharo; session=100678945");
```

HttpURLConnection 特有方法

```
HttpURLConnection http = (HttpURLConnection) url.openConnection();
http.setRequestMethod("GET");          //设置请求方法（必须全大写）              int code = http.getResponseCode();//获取响应码（如 200, 404）
String msg = http.getResponseMessage();//获取响应消息（如"OK", "Not Found"）     http.disconnect();// 显式断开连接，释放资源
```

四、样例代码与解析

遍历并打印所有 HTTP 响应头【场景】：提供任意一个 URL，在控制台显示输出 HTTP 服务器响应的整个头部。

```
    import java.net.*;          import java.util.*;          public class AllHeadersViewer {     public static void main(String[] args) {
            try { URL u = new URL("http://www.szu.edu.cn");          URLConnection uc = u.openConnection();
                // 遍历所有头字段          int i = 1;
                while (true) {String headerName = uc.getHeaderFieldKey(i);          String headerValue = uc.getHeaderField(i);
                    if (headerName == null && headerValue == null) break; // 结束条件
                    if (headerName == null) {System.out.println("状态行: " + headerValue); // 第一行通常是状态行（如 HTTP/1.1 200 OK）}
                    else {          System.out.println(headerName + ": " + headerValue);          }
                    i++;          }          }
            catch (Exception e) {          e.printStackTrace();          }     }          }
```

获取二进制文件并保存在磁盘中【场景】：下载图片等二进制文件。不能用 openStream()(无法判断停止读取的时机)。必须用 getContentLength()作为终止条件。

```
    import java.net.*;          import java.io.*;          public class BinarySaver {     public static void main(String[] args) {
            try { URL u = new URL("http://example.com/image.jpg");          URLConnection uc = u.openConnection();
                String contentType = uc.getContentType();          int contentLength = uc.getContentLength();
                // 这里注意：判断是否为文本或长度未知，若是则抛出异常或做其他处理
                if (contentType.startsWith("text/") || contentLength == -1) { throw new IOException("This is not a binary file or length unknown."); }
                try (InputStream raw = uc.getInputStream()) {
                    InputStream in = new BufferedInputStream(raw);          byte[] data = new byte[contentLength]; // 按 Content-Length 分配数组
                    int offset = 0;          // 循环读取，offset 用来计数，保证按照规定字节数读取
                    while (offset < contentLength) {          int bytesRead = in.read(data, offset, data.length - offset);
                        if (bytesRead == -1) break; // 流意外结束          offset += bytesRead;          }
                    // 提取文件名并保存
                    String filename = u.getFile();          filename = filename.substring(filename.lastIndexOf('/') + 1);
                    try (FileOutputStream fout = new FileOutputStream(filename)) {fout.write(data);fout.flush(); // 这里注意：关闭前必须 flush}}}
            catch (Exception e) {          e.printStackTrace();          }     }          }
```

使用 POST 向服务器提交表单【场景】：向 HTTP 服务器提交表单数据（名值对）。

```
    import java.net.*;          import java.io.*;          public class FormPoster {     public static void main(String[] args) {
            try { URL u = new URL("http://www.example.com/api/submit");          HttpURLConnection uc = (HttpURLConnection) u.openConnection();
                uc.setDoOutput(true);          uc.setRequestMethod("POST");          // 1. 允许输出（POST 必须）
                uc.setRequestProperty("Content-Type", "application/x-www-form-urlencoded");          // 2. 设置请求头
                // 3. 构建并编码表单数据（名 1=值 1&名 2=值 2）
                String formData = "name=" + URLEncoder.encode("张三", "UTF-8")+"&email=" + URLEncoder.encode("test@szu.edu.cn", "UTF-8");
                uc.setRequestProperty("Content-Length", String.valueOf(formData.length()));
                // 4. 获取输出流并写入数据
                try (OutputStream raw = uc.getOutputStream();          DataOutputStream out = new DataOutputStream(raw)) {
                    out.writeBytes(formData);          out.flush();          }// 这里注意：流关闭时会自动 flush，确保数据发送
                // 5. 读取服务器响应
                int responseCode = uc.getResponseCode();
                if (responseCode == HttpURLConnection.HTTP_OK) {
                    try (BufferedReader reader = new BufferedReader(new InputStreamReader(uc.getInputStream(), "UTF-8"))) {
                        String line;          while ((line = reader.readLine()) != null) {System.out.println(line);}}}
                uc.disconnect();     }
            catch (Exception e) {          e.printStackTrace();          }     }          }
```

带错误流处理的健壮下载器（应对 404 等异常）【场景】：下载网页内容，并妥善处理 404 等错误响应，避免程序直接崩溃。

```
    import java.net.*;          import java.io.*;          public class RobustDownloader {     public static void main(String[] args) {
            try { URL u = new URL("http://www.example.com/not-exist.html");          HttpURLConnection conn = (HttpURLConnection) u.openConnection();
                conn.setRequestMethod("GET");          int responseCode = conn.getResponseCode();          System.out.println("响应码: "+responseCode);
                InputStream in;
                // 判断响应码，决定使用正常流还是错误流
                if (responseCode >= 400) {   in = conn.getErrorStream(); // 获取服务器返回的错误提示 HTML          }
                else {          in = conn.getInputStream(); // 获取正常的响应内容          }
                if (in != null) {try (BufferedReader reader = new BufferedReader(new InputStreamReader(in, "UTF-8")))
                    {   String line;          while ((line = reader.readLine()) != null) {System.out.println(line);}          }          }
                conn.disconnect();          }
            catch (IOException e) {          e.printStackTrace();          }     }          }
```

五、一些综合题

简述 Java 中实现 Web 系统级缓存的架构与步骤

【问题】：使用 Java 编程实现具有缓存功能的 HTTP 客户端，需要使用哪些类？如何编程实现？
【解答】：JVM 只支持一个共享缓存。要在 Java 客户端实现系统级缓存，必须自定义以下 3 个抽象类的子类，并注册到系统中：
(1) ResponseCache（缓存管理器）：代表 URLConnection 缓存的实现，决定哪些资源应该被缓存以及缓存多长时间。核心方法是 get()（检索缓存）和 put()
（决定是否存入缓存）。
(2) CacheRequest（缓存写入通道）：表示在 ResponseCache 中存储资源的通道。其 getBody() 方法返回一个 OutputStream，HTTP 客户端调用它来将
资源数据以字节形式写入缓存库。
(3) CacheResponse（缓存读取通道）：表示在 ResponseCache 中检索资源的通道。其 getHeaders() 返回响应头，getBody() 返回响应主体（InputStream）。
【注册步骤】：在程序启动时，实例化自定义的 ResponseCache 子类，并调用 ResponseCache.setDefault(自定义子类对象) 向系统注册。

猜测 MIME 媒体类型

【背景】：并非所有服务器（如老旧的 FTP 服务器）都会正确提供 Content-Type 头。
【解决方法】：URLConnection 提供了两个静态方法来猜测 MIME 类型：
(1) guessContentTypeFromName(String name)：根据文件名后缀猜测（优先使用此方法）。
(2) guessContentTypeFromStream(InputStream in)：根据流中前几个字节的数据特征猜测（要求流支持 mark/reset 操作）。

第八章：客户端 Socket（Client Socket）

一、核心概念

Socket（套接字）的本质：Socket 是两个主机之间的一个网络连接。连接成功后，可以从 Socket 获取输入流和输出流，进行全双工（同时收发）通信。
【协议依赖】：Socket 本身只负责传输字节流，传输数据的含义取决于应用层协议（HTTP、Telnet、Time 协议等）。程序必须自己负责数据的组装、发送和解析。

二、构造与连接 Socket

常用构造方法（连接并指定远程/本地信息）

【Socket(String host, int port)】：连接指定主机和端口。
【Socket(InetAddress host, int port)】：通过 InetAddress 对象连接。
【Socket(String host, int port, InetAddress localAddr, int localPort)】：当主机有多个网卡时，可指定使用哪个本地网卡（localAddr）和本地端口（localPort，
0 表示自动随机分配）。
【Socket(Proxy proxy)】：通过指定的代理服务器进行连接。

先构造后连接（适用于需要设置选项或超时的场景）

【步骤】：先使用无参构造函数创建 Socket，设置好相关选项后，再调用 connect 方法。

```
Socket socket = new Socket();
SocketAddress address = new InetSocketAddress("www.example.com", 80);
socket.connect(address, 5000); // 设置连接超时时间为 5000 毫秒
```

获取 Socket 信息

【getInetAddress() / getPort()】：获取远程主机的 IP 地址和端口号。
【getLocalAddress() / getLocalPort()】：获取本地主机的 IP 地址和端口号。

三、Socket 选项

Socket 提供了 9 个选项来调整底层 TCP/IP 的行为，必须在连接后调用对应的 set 方法。
SO_TIMEOUT (setSoTimeout)　【作用】：设置 read()方法等待数据的最大毫秒数。
【这里注意】：超时不会断开连接，而是抛出 SocketTimeoutException（属于 InterruptedIOException 的子类），程序可捕获异常后继续下一次 read()。
默认值为 0（无限等待）。
SO_KEEPALIVE (setKeepAlive)　【作用】：防止客户端不知道服务器已经崩溃。
【机制】：开启后，客户端每隔两小时通过空闲连接发送探测数据包。若服务器不响应，客户端会持续尝试 12 分钟，若仍无响应则关闭 Socket。默认关闭。
TCP_NODELAY (setTcpNoDelay)
【作用】：禁用 Nagle 算法。开启后，数据包会尽快发送，不考虑包的大小，适用于对实时性要求高的场景（如在线游戏、远程登录）。
SO_LINGER (setSoLinger)　【作用】：控制 Socket 关闭时如何处理尚未发送的数据。
【机制】：默认情况 close()立刻返回，数据在后台继续发送。若开启并设置秒数，close()会阻塞等待指定秒数直到数据发送并收到确认，时间一到则强制关闭。
SO_RCVBUF / SO_SNDBUF (setReceiveBufferSize / setSendBufferSize)
【作用】：设置接收和发送缓冲区的大小（单位为字节）。数据量大且网速快时使用大缓冲区。
OOBINLINE (setOOBInline) 与 sendUrgentData
【作用】：处理带外数据（紧急数据，如 Ctrl+C）。开启后，紧急数据会混入普通输入流中，需要应用程序特别处理。
这里注意，每次只能发送 1 个字节的紧急数据。
SO_REUSEADDR (setReuseAddress)　【作用】：允许绑定处于 TIME_WAIT 状态的端口，常用于服务器快速重启。
【这里注意】：必须先使用无参构造函数创建 Socket，设置该选项为 true 后，再调用 connect()或 bind()才能生效。
IP_TOS (setTrafficClass)　【作用】：指定数据包的优先级（服务类型），与延时和丢包率相关。

四、 Socket 异常处理

在构造或连接 Socket 时，可能会抛出以下 IOException 的子类异常：

BindException：本地端口被占用，或无权限使用该端口。　**ConnectException**：连接被远程主机拒绝（通常是远程主机没在监听对应端口，或进程忙）。

NoRouteToHostException：连接超时或网络不可达。　**ProtocolException**：数据不符合 TCP/IP 协议规范。

五、 半关闭与状态判断

半关闭 Socket　【方法】：shutdownInput() 和 shutdownOutput()。

【作用】：仅关闭连接的一半（输入或输出流），使流认为到了末尾，但并未释放 Socket 关联的底层资源（如端口）。

【这里注意】：即使半关闭了连接，使用结束后仍必须调用 close()来彻底释放资源。关闭输入流后再读取会返回-1，关闭输出流后再写入会抛出 IOException。

判断 Socket 状态　【易错点】：isConnected()方法只要曾经连接过就返回 true，即使后来断开了也返回 true。

【正确逻辑】：判断一个 Socket 当前是否处于"打开且已连接"状态，必须结合两个方法：boolean isOpen = socket.isConnected() && !socket.isClosed();

六、 代码

基础连接与流获取

```
Socket socket = new Socket("time.nist.gov", 13);
InputStream in = socket.getInputStream();    OutputStream out = socket.getOutputStream();
```

设置超时与读取

```
socket.setSoTimeout(15000); // 15 秒超时
InputStreamReader reader = new InputStreamReader(in, "ASCII");
int c;
while ((c = reader.read()) != -1) {/*  处理字符*/}
```

发送 HTTP GET 请求（结合 Socket 与半关闭）

```
Writer out = new OutputStreamWriter(socket.getOutputStream(), "ISO-8859-1");
out.write("GET / HTTP/1.0\r\n\r\n");
out.flush();
socket.shutdownOutput(); //  半关闭输出流，通知服务器请求发送完毕
```

七、 样例代码与解析

Daytime 协议客户端（时间服务器）【场景】连接 NIST 时间服务器（端口 13），读取并打印服务器返回的 ASCII 时间字符串，处理超时异常。

```
import java.net.*;        import java.io.*;        public class DaytimeClient {    public static void main(String[] args) {
        Socket socket = null;
        try { socket = new Socket("time.nist.gov", 13);        socket.setSoTimeout(15000); //  设置读取超时时间为 15 秒，防止服务器挂起导致程序死锁
            InputStream in = socket.getInputStream();        InputStreamReader reader = new InputStreamReader(in, "ASCII");
            StringBuilder time = new StringBuilder();        int c;        while ((c = reader.read()) != -1) {time.append((char) c);}
            System.out.println(time.toString());        }
        catch (SocketTimeoutException ex) {        System.err.println("读取数据超时，服务器可能已挂起。");        }
        catch (IOException ex) {        System.err.println("连接或读取失败: " + ex.getMessage());        }
        finally {        if (socket != null) {try { socket.close(); } catch (IOException e) { /*  忽略  */ }}    }    }    }
```

使用 Socket 发送 HTTP GET 请求并半关闭【场景】通过 Socket 直接向 Web 服务器发送 HTTP 请求，获取响应。使用半关闭通知服务器数据发送完毕。

```
import java.net.*;        import java.io.*;        public class SimpleHTTPGet {    public static void main(String[] args) {
        try (Socket connection = new Socket("www.example.com", 80)) {        // 1. 获取输出流并发送 HTTP 请求头
            Writer out = new OutputStreamWriter(connection.getOutputStream(), "ISO-8859-1");        out.write("GET / HTTP/1.0\r\n");
            out.write("Host: www.example.com\r\n");        out.write("\r\n"); //  空行表示请求头结束        out.flush();
            connection.shutdownOutput();        // 2. 半关闭输出流，告诉服务器请求已发送完毕，可以返回响应了
            // 3. 获取输入流并读取服务器响应
            BufferedReader reader = new BufferedReader(new InputStreamReader(connection.getInputStream(), "UTF-8"));        String line;
            while ((line = reader.readLine()) != null) {System.out.println(line);}        }
        catch (IOException ex) {        ex.printStackTrace();        }    }    }
```

端口扫描器（利用 ConnectException 判断端口开放）【场景】检查本地主机（127.0.0.1）的前 1024 个端口中，哪些正在运行 TCP 服务器。

```
import java.net.*;        import java.io.*;        public class LowPortScanner {    public static void main(String[] args) {
        String host = "localhost";        if (args.length > 0) host = args[0];
        for (int i = 1; i <= 1024; i++) {
            try (Socket socket = new Socket(host, i)) {System.out.println("端口  " + i + "  是开放的。");}//  如果能成功构造 Socket，说明端口开放
            catch (ConnectException e) {        //  连接被拒绝，说明端口未开放或无服务监听        }
            catch (IOException e) {        //  其他异常（如超时）        }        }    }}
```

第九章：服务端 ServerSocket（Server Socket）

一、 核心概念通俗理解

ServerSocket 与 Socket 的区别

【Socket】：客户端用来主动连接服务器的"电话机"。

【ServerSocket】：服务器端用来监听端口、等待客户端连接的"总机"。它不直接参与数据传输，当有客户端连接时，它会生成一个普通的 Socket 对象，由这个 Socket 对象负责与客户端进行数据交互。

【关系】：ServerSocket 和 Socket 没有继承关系，它们是合作关系。

服务器架构的演进

【单线程迭代服务器】：一次只能处理一个客户端，其他客户端须在队列中等待。如任务简单且快（如 Daytime 时间协议）可以使用；否则导致严重的阻塞。

【多线程服务器】：为每个连接的客户端分配一个独立线程。优点并发能力强；缺点线程创建和销毁开销大，若连接数过多会导致系统资源耗尽（DoS 攻击）。

【线程池服务器】：使用固定大小的线程池处理连接。将客户端连接包装成任务提交给线程池，防止资源耗尽，速度更快，是实际开发中最常用的模式。

二、 猜测考点

ServerSocket 的状态判断

【注意】：判断一个 ServerSocket 当前是否处于"打开且已绑定端口"的状态，不能只用 isBound()。

【正确逻辑】：必须结合两个方法：boolean isOpen = ss.isBound() && !ss.isClosed();

构造但不绑定（高级用法）

【场景】：如果需要在绑定端口前设置某些 Socket 选项（如 SO_REUSEADDR），必须先使用无参构造函数创建 ServerSocket，设置好选项后，再调用 bind()方法绑定端口。

【代码体现】：

```
ServerSocket ss = new ServerSocket();                ss.setReuseAddress(true);
SocketAddress addr = new InetSocketAddress(80);        ss.bind(addr);
```

服务器日志（Logger）

【核心原则】：系统上线后，不应输出调试信息（如 FINE、FINER、FINEST 级别），只记录请求和服务器错误。

【必备信息】：日志中一般必须包含"时间戳"和"客户端地址"。

【日志级别（从高到低）】：SEVERE（严重错误），WARNING, INFO, CONFIG, FINE, FINER, FINEST。

ServerSocket 特有选项

【SO_TIMEOUT】：这里注意，对 ServerSocket 而言，它影响的是 accept()方法的阻塞等待时间。超时抛出 SocketTimeoutException。

【SO_REUSEADDR】：允许绑定处于 TIME_WAIT 状态的端口，常用于服务器快速重启。

【setPerformancePreferences】：设置服务器对高带宽、低延迟、高吞吐量的重视程度。

三、 代码

构造与绑定

```
ServerSocket ss1 = new ServerSocket(8080); //  直接绑定        ServerSocket ss2 = new ServerSocket(8080, 50); //  指定 backlog（连接队列长度）
InetAddress local = InetAddress.getByName("192.168.1.100");        ServerSocket ss3 = new ServerSocket(8080, 50, local); //  绑定特定网卡 IP
```

获取 ServerSocket 信息

```
int port = ss.getLocalPort(); //  获取绑定的本地端口        InetAddress addr = ss.getInetAddress(); //  获取绑定的本地 IP
                                                            SocketAddress sAddr = ss.getLocalSocketAddress(); //  获取完整地址
```

日志记录标准写法

```
private final static Logger logger = Logger.getLogger("requests");
logger.info(new Date() + " " + connection.getRemoteSocketAddress());//  记录客户端连接信息
logger.log(Level.SEVERE, "unexpected error " + ex.getMessage(), ex); //  记录严重错误
```

四、 样例代码与解析

基础迭代时间服务器（单线程骨架）【场景】：遵循 RFC868 时间协议，客户端连接后，服务器发送当前时间并关闭连接。

```
import java.net.*;        import java.io.*;        import java.util.Date;        public class TimeServer {
    public static final int PORT = 8080;
    public static void main(String[] args) {        //  使用 try-with-resources 自动关闭 ServerSocket
        try (ServerSocket server = new ServerSocket(PORT)) {
            while (true) {        // accept()会阻塞，直到有客户端连接
                try (Socket connection = server.accept()) {
                    Writer out = new OutputStreamWriter(connection.getOutputStream());        Date now = new Date();
                    out.write(now.toString() + "\r\n");        out.flush();        //  连接结束，Socket 自动关闭        }
                catch (IOException ex) {        //  处理单个连接的异常，不影响服务器继续监听        }        }        }
        catch (IOException ex) {        ex.printStackTrace();        }    }    }
```

线程池时间服务器（大题编程骨架）【场景】：将客户端连接交给线程池处理，防止并发过高导致服务器崩溃。

```
import java.net.*;        import java.io.*;        import java.util.Date;        import java.util.concurrent.*;    public class PooledTimeServer {
    public static final int PORT = 8080;    public static final int THREAD_COUNT = 4; //  线程池大小 4
    public static void main(String[] args) {ExecutorService pool = Executors.newFixedThreadPool(THREAD_COUNT);        // 1. 创建线程池
        try (ServerSocket server = new ServerSocket(PORT)) {
            while (true) {Socket connection = server.accept();
                // 2. 将连接包装成任务，提交给线程池
                Callable<Void> task = new TimeTask(connection);        pool.submit(task);        }        }
        catch (IOException ex) {        ex.printStackTrace();        }
        pool.shutdown();    }
    // 3. 定义处理客户端连接的任务类
    private static class TimeTask implements Callable<Void> {
        private Socket connection;        TimeTask(Socket connection) {this.connection = connection;}
        @Override
        public Void call() {
            try (Socket conn = this.connection) { Writer out = new OutputStreamWriter(conn.getOutputStream());
                Date now = new Date();        out.write(now.toString() + "\r\n");        out.flush();        }
            catch (IOException ex) {        ex.printStackTrace();        }
            return null;        }    }    }
```

带日志记录的服务器【场景】：在服务器启动和客户端连接时记录日志。

```
import java.net.*;        import java.io.*;        import java.util.Date;        import java.util.logging.*;    public class LoggingServer {
    private final static Logger logger = Logger.getLogger("requests");//  创建 Logger 对象
    public static void main(String[] args) {
        try (ServerSocket server = new ServerSocket(8080)) {
            logger.info("Server started on port " + server.getLocalPort());
            while (true) {Socket connection = server.accept();        //  记录客户端连接的时间和地址
                logger.info(new Date() + " " + connection.getRemoteSocketAddress());    //  交由线程处理...        }        }
        catch (IOException ex) {        //  记录严重错误，包含异常堆栈    logger.log(Level.SEVERE, "unexpected error " + ex.getMessage(), ex);}}}
```

五、 综合题相关

简述多线程服务器的编程实现及线程池的作用。

【解答】：(1) 编程实现：服务器端使用 ServerSocket 监听端口，调用 accept()方法获取与客户端连接的 Socket 对象。为了支持多用户并发，不为每个连接创建新进程，而是将 Socket 对象传递给一个实现了 Runnable 或 Callable 接口的线程类，调用 start()方法启动新线程处理该连接的数据交互。

(2) 线程池的作用：频繁创建和销毁线程会消耗大量系统资源。使用线程池（如 ExecutorService）可以复用已创建的线程，降低开销；同时能限制同一时刻运行的最大线程数，防止因并发连接过多导致内存耗尽或系统崩溃（防御 DoS 攻击），提高服务器的稳定性和响应速度。

如何判断一个 ServerSocket 当前处于正常打开且已绑定端口的状态？

【解答】：不能仅依靠单一方法。isBound()返回 true 表示已绑定端口，但可能随后被关闭；isClosed()返回 true 表示已关闭。因此，正确的判断逻辑是同时满足已绑定且未关闭：boolean isOpen = serverSocket.isBound() && !serverSocket.isClosed();

第十章：安全 Socket（Secure Socket）

一、 核心概念

加密算法的两种基本类型

【对称加密（秘密密钥加密）】：加密方和解密方使用相同的密钥。优点是速度快，适合加密大量数据；缺点是密钥分发困难。

【非对称加密（公开密钥加密）】：加密方和解密方使用不同的密钥。分为公开密钥（公钥，用于加密）和私有密钥（私钥，用于解密）。优点是安全性高，适合身份认证和密钥交换；缺点是速度慢。

SSL/TLS 中的组合应用【实际应用场景】：为了兼顾效率与安全，HTTPS 等安全协议通常采用"组合拳"：

(1) 使用非对称加密来传输"对称加密的密钥"（解决密钥分发问题）。(2) 使用协商好的"对称加密密钥"来传输实际的业务数据（解决速度问题）。

(3) 使用非对称加密进行身份认证和消息完整性检查。

中间人攻击（Man-in-the-Middle Attack）

【原理】：攻击者在通信的两端之间插入自己的设备或程序，拦截、篡改或伪造通信数据。例如，客户端以为自己在和服务器通信，实际上是在和攻击者通信；攻击者再伪装成客户端和服务器通信。

【防范机制】：引入可信任的第三方认证机构（CA）。服务器的公钥必须放在由 CA 颁发的数字证书中，客户端通过验证证书来确认服务器的真实身份，从而防止中间人伪造公钥。

二、 猜测考点

安全 Socket 的创建方式

【这里注意】：不能使用 new Socket() 或 java.net.Socket 直接创建安全 Socket 对象。必须通过工厂类 SSLSocketFactory.getDefault() 获取工厂实例，再调用 createSocket() 方法创建。返回的类型是 javax.net.ssl.SSLSocket，它是 Socket 的子类。

密码组（Cipher Suites）的命名规则

【概念】：JSSE 支持认证和加密的不同组合，称为密码组。

【命名结构】：算法名称通常分为 4 个部分（以 SSL_DH_anon_EXPORT_WITH_DES40_CBC_SHA 为例）：

(1) 协议版本：SSL（安全套接字层）版本 3。　(2) 密钥交换算法：DH（Diffie-Hellman）。

(3) 身份认证与加密算法：anon（无身份认证）、EXPORT（出口级）、DES40（40 位密钥的数据加密标准算法）。

(4) 校验和（MAC）：CBC（密码块链）、SHA（安全散列算法）。

【猜测考点】：如果考题问 SSL_DH_anon_EXPORT_WITH_DES40_CBC_SHA 中的 DES40 表示什么？答案是：加密算法。

默认启用的密码组限制

【安全原则】：JDK 默认支持并启用前 28 个相对安全的密码组。

【这里注意】：对于无认证（anon）或认证但不加密的事务，必须使用 setEnabledCipherSuites() 方法显式启用。在编程时，应尽量避免使用名字中包含 NULL、ANON、EXPORT 的密码组，因为它们存在严重的安全漏洞。

客户端模式与身份认证

【setUseClientMode(boolean)】：确定是否需要在第一次握手时使用认证。True 表示 Socket 处于客户端模式（不会向对方提供自行认证），False 表示会自行认证。该属性对指定的 Socket 只能设定一次。

【setNeedClientAuth(boolean)】：服务器端安全 Socket 使用此方法，要求与他连接的所有客户端都必须自行认证。

三、 代码

创建安全 Socket 与安全 ServerSocket

```
//  创建安全客户端 Socket
SocketFactory factory = SSLSocketFactory.getDefault();        Socket socket = (SSLSocket) factory.createSocket("login.ibiblio.org", 7000);
//  创建安全服务器端 ServerSocket
ServerSocketFactory ssf = SSLServerSocketFactory.getDefault();        ServerSocket ss = ssf.createServerSocket(443);
```

密码组的管理方法

```
String[] supported = socket.getSupportedCipherSuites();//  获取该 Socket 上所有可用的算法组合
String[] enabled = socket.getEnabledCipherSuites();//  获取当前 Socket 上实际启用的密码组
socket.setEnabledCipherSuites(supported); //  指定/修改当前 Socket 使用哪些密码组，例如启用所有支持的
```

会话管理与事件监听

```
SSLSession session = ((SSLSocket) socket).getSession();//  获取 Socket 所属的会话（用于复用密钥，提升性能）
((SSLSocket) socket).addHandshakeCompletedListener(event->{System.out.println("握手√，用密码组:"+event.getCipherSuite());});//注册握手完成事件监听器
```

四、 样例代码与解析

获取并打印安全 Socket 支持的密码组（随堂测试 3 编程题原题）【场景】：编写一个名为 CipherSuites 的类，生成安全 Socket（协议为 HTTPS，端口为 443），按行打印出当前 Socket 所支持的所有密码组，并输出密码组的总个数。

```
import java.io.*;        import javax.net.ssl.*;        public class CipherSuites {    public static void main(String[] args) {
        try {SSLSocketFactory factory = (SSLSocketFactory) SSLSocketFactory.getDefault();// 1. 获取默认的 SSL Socket 工厂
            SSLSocket socket = (SSLSocket) factory.createSocket("www.example.com", 443); //2.创建安全 Socket(连接任意支持 HTTPS 的服务器如 443 端口)
            String[] cipherSuites = socket.getSupportedCipherSuites();// 3. 获取支持的所有密码组
            // 4. 按行打印并统计个数
            System.out.println("支持的密码组列表:");        for (String suite : cipherSuites) {        System.out.println(suite);        }
            System.out.println("密码组的总个数: " + cipherSuites.length);
            socket.close();// 5. 关闭 Socket        }
        catch (IOException e) {        e.printStackTrace();        }    }        }
```

启用不安全的匿名密码组【场景】：在 SSLServerSocket 上启动匿名的非认证连接（查找密码组包含 anon 的组）。

```
//  假设  server  是一个  SSLServerSocket  对象
String[] supported = server.getSupportedCipherSuites();    String[] safe = new String[supported.length];    int count = 0;
//  筛选出包含  "anon"  的密码组
for (String suite : supported) {    if (suite.indexOf("_anon_") > 0) {        safe[count++] = suite;    }}
//  创建实际要启用的数组
String[] enabled = new String[count];        System.arraycopy(safe, 0, enabled, 0, count);
server.setEnabledCipherSuites(enabled); //  显式启用这些密码组
```

五、 综合题相关

简述什么是中间人攻击，如何有效解决？（随堂测试 3 简答题原题）

【解答】：

(1) 概念：中间人攻击（Man-in-the-Middle Attack）是一种常见的网络安全威胁。攻击者在通信的两端之间插入自己的设备或程序，拦截、窃取、篡改或伪造通信数据。通信双方误以为自己在直接对话，实际上数据都经过了攻击者的中转。

(2) 有效解决措施：

使用加密通信：采用 SSL/TLS（如 HTTPS）等加密协议，确保数据在传输过程中是加密的，即使被拦截也无法读取明文。

证书与公钥基础设施（PKI）：使用数字证书和 PKI 来验证通信实体的身份。服务器的公钥必须存放在由受信任的第三方认证机构（CA）颁发的证书中，客户端通过验证证书来防止攻击者伪造公钥。

安全的网络环境：避免在不受信任的公共 Wi-Fi 下进行敏感操作。

第十一章：非阻塞 I/O（NIO）

一、 核心概念

阻塞 I/O 与非阻塞 I/O 的区别

【阻塞 I/O（Blocking IO）】：调用 read() 时，如果没有数据，当前线程会被挂起等待，直到有数据可读才返回。就像小明找小刚借书，小刚去找书，小明只能在原地干等。

【非阻塞 I/O（Non-blocking IO）】：调用 read() 时，无论是否有数据都会立刻返回。就像小明找小刚借书，小刚说"我得找一会"，小明就去干别的事了，过会再来拿。线程不会被阻塞。

传统 IO 与 NIO 的区别

【传统 IO】：面向流（Stream），一次处理一个或几个字节，无法直接操作缓冲区。

【NIO（New IO）】：面向缓冲区（Buffer），数据总是先被读入缓冲区，或从缓冲区写入。支持阻塞和非阻塞两种模式。

NIO 的核心组件

【Channel（通道）】：表示 IO 源与目标打开的连接，类似于传统的流，但 Channel 本身不能直接访问数据，只能与 Buffer 进行交互。

【Buffer（缓冲区）】：用于特定基本数据类型的容器。　【Selector（选择器）】：允许单线程处理多个 Channel，通过注册事件来监听多个通道的状态。

二、 猜测考点

Buffer 的四个关键属性

【Position（位置）】：下一个将要被写入或读取的元素索引。　【Capacity（容量）】：缓冲区可保存元素的最大数目（底层数组的大小）。

【Limit（限度）】：还有多少数据需要取出（读模式），或还有多少空间可以放入数据（写模式）。不能超过容量。

【Mark（标记）】：客户端指定的索引，用于记录某个 position，方便后续通过 reset() 恢复。

【大小关系】：这里注意，任何时候都必须满足 0 <= position <= limit <= capacity。

Buffer 的状态切换方法

【flip()】：从写模式切换到读模式。将 limit 设置为当前的 position，将 position 设置为 0，mark 设为 -1。

【clear()】：清空缓冲区，准备重新写入。将 position 设置为 0，将 limit 设置为 capacity。这里注意，它并不删除实际数据，仅仅改变指针。

【rewind()】：将 position 设置为 0，不改变 limit，取消 mark。可用于反复读取缓冲区中的数据。

【compact()】：将 position 与 limit 之间的数据复制到 buffer 的开始位置，并将 limit - position 赋值给 position，将 capacity 赋值给 limit。常用于非阻塞模式下，保留未读完的数据继续读取。

非阻塞 I/O 的优缺点与适用场景

【优点】：单线程可以处理多个连接（通过 Selector）。　【缺点】：一般情况下，性能不如"多线程 + 阻塞 IO"。

【适用场景】：连接数很多，但每个连接并不活跃（如长连接心跳检测）。

【默认状态】：这里注意，默认情况下，SocketChannel 和 ServerSocketChannel 对象处于阻塞模式，需调用 configureBlocking(false) 切换为非阻塞。

三、 代码

Channel 的打开与配置

```
//  打开 SocketChannel 并连接
SocketAddress addr = new InetSocketAddress("www.example.com", 80);        SocketChannel client = SocketChannel.open(addr);
client.configureBlocking(false); //  切换为非阻塞模式
```

Buffer 的创建与数据操作

```
ByteBuffer buffer = ByteBuffer.allocate(100); //  分配容量为 100 的 ByteBuffer
//  包装现有数组（慎用，共享底层数组）
byte[] data = "Hello".getBytes();        ByteBuffer buffer2 = ByteBuffer.wrap(data);
//  写入数据
buffer.put('H');        buffer.put("ello".getBytes());
buffer.flip();//  切换为读模式
byte b = buffer.get();//  读取数据
```

Selector 的注册与事件监听

```
Selector selector = Selector.open();
//  注册通道及感兴趣的事件（如接收连接、可读、可写）
serverChannel.register(selector, SelectionKey.OP_ACCEPT);        clientChannel.register(selector, SelectionKey.OP_READ);
//  阻塞等待事件发生
selector.select();        Set<SelectionKey> readyKeys = selector.selectedKeys();
```

四、 样例代码与解析

NIO 客户端：阻塞模式与非阻塞模式对比【场景】：从服务器获取字符，并打印到控制台。

```
import java.net.*;        import java.nio.*;        import java.nio.channels.*;        public class NIOClientExample {
    public static void main(String[] args) throws Exception {
        SocketAddress addr = new InetSocketAddress("localhost", 19);        SocketChannel client = SocketChannel.open(addr);
        ByteBuffer buffer = ByteBuffer.allocate(74);        WritableByteChannel output = Channels.newChannel(System.out);
        //  【阻塞模式】：read()会一直等待直到读到数据或流结束
/*        while (client.read(buffer) != -1) {        buffer.flip();        output.write(buffer);        buffer.clear();        }        */
        //  【非阻塞模式】：read()立刻返回，需判断返回值
        client.configureBlocking(false);
        while (true) {int n = client.read(buffer);
            if (n > 0) {        buffer.flip();        output.write(buffer);        buffer.clear();        }
            else if (n == -1) {   break; //  流结束  }        //  如果  n == 0，说明没有数据，继续循环（实际应用中可配合 Selector 避免空转）        }
        client.close();    }}
```

NIO 服务器端：基于 Selector 的单线程模型【场景】：使用单线程处理多个客户端的连接和读写请求。

```
import java.net.*;        import java.nio.*;        import java.nio.channels.*;        import java.util.*;        public class NIOServerExample {
    public static void main(String[] args) throws Exception {
        ServerSocketChannel serverChannel = ServerSocketChannel.open();        serverChannel.bind(new InetSocketAddress(19));
        serverChannel.configureBlocking(false); //  必须设为非阻塞才能注册到 Selector
        Selector selector = Selector.open();        serverChannel.register(selector, SelectionKey.OP_ACCEPT);
        while (true) {
            selector.select(); //  阻塞等待，直到有注册的事件发生
            Set<SelectionKey> readyKeys = selector.selectedKeys();        Iterator<SelectionKey> iterator = readyKeys.iterator();
            while (iterator.hasNext()) {
                SelectionKey key = iterator.next();        iterator.remove(); //  必须手动移除已处理的 key
                if (key.isAcceptable()) {        SocketChannel clientChannel = serverChannel.accept();
                    clientChannel.configureBlocking(false);        clientChannel.register(selector, SelectionKey.OP_WRITE);        }
                else if (key.isWritable()) {        SocketChannel clientChannel = (SocketChannel) key.channel();
                    clientChannel.write(ByteBuffer.wrap("Hello NIO".getBytes()));        //  执行写入操作...
                    key.cancel(); //  写入完成后取消监听        clientChannel.close();   }   }   }   }   }
```

五、 综合题相关

简述 NIO 中 Buffer 的状态切换过程（以写数据后读取为例）

【解答】：

(1) 初始状态：创建 Buffer 后，处于写模式，position=0，limit=capacity。　(2) 写入数据：通过 put() 写入数据，position 随之增加，limit 保持不变。

(3) 切换读模式：调用 flip() 方法，将 limit 设置为当前的 position（即有效数据的末尾），将 position 重置为 0。此时进入读模式。

(4) 读取数据：通过 get() 读取数据，position 随之增加，直到等于 limit。

(5) 清空准备重写：调用 clear() 方法，将 position 设为 0，limit 设为 capacity，准备下一轮写入。

第十二章：UDP 数据报（UDP Datagram）

一、 核心概念

UDP 与 TCP 的核心区别

【TCP】：面向连接、可靠、保证顺序、速度慢。适用于 HTTP、FTP 等需要准确无误的场景。

【UDP】：无连接、不可靠、不保证顺序、速度快。适用于 DNS、视频流、实时游戏、NFS 等对速度要求高、允许少量丢包的场景。

【本质差异】：TCP 协议使用的是 Socket 和 ServerSocket，基于"流（Stream）"进行通信；而 UDP 协议没有连接概念，不支持流，它只是"数据包（Datagram）"的发送和接收。

UDP 通信的两个核心组件

【DatagramPacket（数据报包）】：相当于"快递包裹"，负责对数据进行包装，包含数据本身、目标/源 IP 地址和端口号。

【DatagramSocket（数据报套接字）】：相当于"快递收发站"，负责发送和接收 DatagramPacket。

二、 猜测考点

数据包的大小限制

【考点】：大部分系统支持的数据包大小为 8KB（8192 字节）。

【这里注意】：虽然 IPv4 的理论最大值为 65507 字节（IPv6 为 65536 字节），但实际编程中一般不要超过 8KB，否则可能会在网络层被分片甚至丢弃。

发送端与接收端的 DatagramPacket 构造差异

【接收端构造】：只需要提供 byte 数组和长度。因为接收时不知道对方发来的数据有多大，需要预先分配一个足够大的缓冲区（如 1024 字节）去接。

【发送端构造】：除了 byte 数组和长度，还必须明确指定目标 InetAddress 和端口号。

DatagramSocket 的常用选项

【SO_TIMEOUT】：设置接收数据时的超时时间。　【SO_REUSEADDR】：允许多个 DatagramSocket 绑定到同一个网络接口和端口（与 TCP 类似）。

【SO_BROADCAST】：开启广播发送/广播接收功能。这里注意，不要直接使用广播地址进行广播，需要开启此选项。

三、 代码

创建 DatagramSocket

```
DatagramSocket serverSocket = new DatagramSocket(12345);        //  接收端：绑定特定端口
DatagramSocket clientSocket = new DatagramSocket();            //  发送端：系统自动分配可用端口
clientSocket.setSoTimeout(10000);                              //  设置超时
```

构造发送数据包（客户端发给服务器）

```
String message = "Hello, Server!";
byte[] sendData = message.getBytes("UTF-8");
InetAddress serverAddress = InetAddress.getByName("localhost");
DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, serverAddress, 12345); //  参数：数据，偏移量，长度，目标 IP，目标端口
clientSocket.send(sendPacket);
```

构造接收数据包（服务器接收数据）

```
byte[] receiveData = new byte[1024];
DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);        //  参数：缓冲区，偏移量，长度
serverSocket.receive(receivePacket);                                                      //  阻塞等待，直到收到数据
```

解析接收到的数据

```
int length = receivePacket.getLength();                                  //  获取实际接收到的字节长度
String receivedMessage = new String(receivePacket.getData(), 0, length); //  将有效字节转换为字符串
InetAddress clientAddress = receivePacket.getAddress();                  //  获取发送方的 IP 和端口，用于回复
int clientPort = receivePacket.getPort();
```

四、 样例代码与解析

基础 UDP 通信（客户端与服务器一问一答）【场景】：客户端发送一句话，服务器接收后回复一句话。

```
import java.io.IOException;        import java.net.*;
public class UDPBasicExample {    public static void main(String[] args) {
        //  启动服务器线程
        new Thread(() -> {
            try { DatagramSocket serverSocket = new DatagramSocket(12345);        byte[] receiveData = new byte[1024];
                DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
                System.out.println("服务器已启动，等待数据...");        serverSocket.receive(receivePacket);
                String clientMsg = new String(receivePacket.getData(), 0, receivePacket.getLength());
                System.out.println("服务器收到: " + clientMsg);
                //  构造回复数据
                String reply = "Hello, Client!";        byte[] sendData = reply.getBytes();
                //  这里注意：回复包的 IP 和端口必须使用接收包中获取的客户端信息
                DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length,receivePacket.getAddress(), receivePacket.getPort());
                serverSocket.send(sendPacket);        serverSocket.close();        }
            catch (IOException e) {        e.printStackTrace();        }        }).start();
        //  客户端逻辑
        try { Thread.sleep(500); //  等待服务器启动
            DatagramSocket clientSocket = new DatagramSocket();        String message = "Hello, Server!";
            byte[] sendData = message.getBytes();        InetAddress serverAddress = InetAddress.getByName("localhost");
            DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, serverAddress, 12345);        clientSocket.send(sendPacket);
            byte[] receiveData = new byte[1024];        DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
            clientSocket.receive(receivePacket);        String serverMsg = new String(receivePacket.getData(), 0, receivePacket.getLength());
            System.out.println("客户端收到回复: " + serverMsg);        clientSocket.close();        }
        catch (Exception e) {        e.printStackTrace();        }    }    }
```

大数据块分片发送（利用 setData 动态修改）【场景】：当要发送的数据大于 8KB 时，不能一次性塞进一个 DatagramPacket，需要分片发送。

```
import java.net.*;        import java.io.*;        public class UDPSendBigData {    public static void main(String[] args) throws Exception {
        DatagramSocket socket = new DatagramSocket();        InetAddress host = InetAddress.getByName("localhost");
        //  假设这是一个很大的字节数组
        byte[] bigarray = new byte[20000];        int offset = 0;        int bytesSent = 0;
        DatagramPacket dp = new DatagramPacket(bigarray, offset, 512, host, 8080);        //  每次最多发送 512 字节
        while (bytesSent < bigarray.length) {
            socket.send(dp);    bytesSent+=dp.getLength();    int bytesToSend=bigarray.length-bytesSent;    int size=(bytesToSend>512) ? 512 : bytesToSend;
            dp.setData(bigarray, bytesSent, size);        //  这里注意：动态修改数据包的数据源和长度        }
        socket.close();    }    }
```

五、 综合题相关

基于 UDP 传输协议使用 Java 编程实现一个 UDP 客户端与服务器端，进行通信。（原题）

【解答思路】：

(1) 服务器端：使用 DatagramSocket 绑定固定端口，创建 byte 数组和 DatagramPacket 用于接收数据，调用 receive()阻塞等待。收到数据后，解析出客户端的 IP 和端口，构造新的 DatagramPacket 进行回复。

(2) 客户端：使用 DatagramSocket（不指定端口，系统自动分配），获取目标服务器的 InetAddress，将数据打包成 DatagramPacket（指定目标 IP 和端口），调用 send()发送。然后创建接收包等待服务器回复。

【代码】：

```
import java.net.*;        import java.io.*;        public class UDPServer {    public static void main(String[] args) {
        try { // 1. 创建服务器端 DatagramSocket，绑定到指定端口（如 9999）
            DatagramSocket serverSocket = new DatagramSocket(9999);        System.out.println("服务器已启动，等待客户端数据...");
            // 2. 准备接收数据的 DatagramPacket（需预分配字节数组）
            byte[] receiveData = new byte[1024];        DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
            serverSocket.receive(receivePacket);        // 3. 接收数据（阻塞方法，直到收到数据包）
            // 4. 解析数据（注意使用 getLength() 获取实际接收到的字节数）
            String receivedMessage = new String(receivePacket.getData(), 0, receivePacket.getLength());
            System.out.println("收到客户端消息: " + receivedMessage);
            // 5. 获取客户端的 IP 地址和端口号（关键：用于向该客户端回复）
            InetAddress clientAddress = receivePacket.getAddress();        int clientPort = receivePacket.getPort();
            // 6. 构造回复消息
            String replyMessage = "你好，服务器已收到你的消息！";        byte[] sendData = replyMessage.getBytes();
            // 7. 构造发送数据包（必须指定目标地址和端口）
            DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, clientAddress, clientPort);
            // 8. 发送回复
            serverSocket.send(sendPacket);        System.out.println("已向客户端发送回复。");
            serverSocket.close();        // 9. 关闭 Socket        }
        catch (IOException e) {        e.printStackTrace();        }    }    }
    import java.net.*;    import java.io.*;        public class UDPClient {    public static void main(String[] args) {
        try {DatagramSocket clientSocket = new DatagramSocket();// 1. 创建客户端 DatagramSocket（不指定端口，系统自动分配）
            // 2. 准备要发送的数据
            String message = "Hello Server, I am Client!";        byte[] sendData = message.getBytes();
            //  获取目标服务器地址（localhost 或实际 IP）
            InetAddress serverAddress = InetAddress.getByName("localhost");        int serverPort = 9999; //  目标服务器端口
            // 3. 构造发送数据包（必须指定目标地址和端口）
            DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, serverAddress, serverPort);
            // 4. 发送数据
            clientSocket.send(sendPacket);        System.out.println("已发送消息给服务器。");
            // 5. 准备接收服务器的回复
            byte[] receiveData = new byte[1024];        DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
            clientSocket.receive(receivePacket);        // 6. 接收回复（阻塞方法）
            // 7. 解析回复数据
            String replyMessage=new String(receivePacket.getData(),0,receivePacket.getLength());        System.out.println("收到回复:"+replyMessage);
            clientSocket.close();        // 8. 关闭 Socket        }
        catch (IOException e) {        e.printStackTrace();        }    }    }
```

利用 UDP 传输图片文件（结合多线程与 I/O 流）【场景】：服务器读取本地图片，分块通过 UDP 发送给客户端，客户端接收并拼接，最后显示。

【关键点】：

(1) 服务器端：使用 FileInputStream 读取图片，循环调用 read(byte[])，每次读取的数据打包成 DatagramPacket 发送。发送完毕后，发送一个特定的结束标志（如文本"end"）。

(2) 客户端：使用 ByteArrayOutputStream 作为内存缓冲区，循环调用 receive()接收数据，判断如果是"end"则跳出循环，否则将数据写入 ByteArrayOutputStream。最后通过 toByteArray()获取完整图片字节数组进行渲染。